



MedVault
**Machine Learning Assisted GDPR Compliance in
Clinical Environments:**
**An Evaluation and Comparative Analysis of 3 Distinct ML Implementations of
Automated Document Classification Within a Serverless Cloud Architecture**

Final Project Report

TU858
BSc in Computer Science (International)

Stephen Thompson
C21394693

Mr Ciaran O'Driscoll

School of Computer Science
Technological University, Dublin

19/04/26

Abstract

The proliferation of data in the modern age has altered the relationship between organisations and the data they hold. Data has become an invaluable asset in modern society with healthcare in Ireland expecting a pivot to serverless paperless technology by 2030 [1].

The 2021 HSE cyberattack which resulted in 470 legal proceedings against the HSE [2] proves our health data is not out of reach for malicious actors. This project's goals are to address support this trend towards paperless healthcare documentation [3] and provide insight into the future of semi-automated document classification by leveraging modern machine learning technologies as well as their viability in a clinical setting by performing a comparative analysis of models within a fully serverless file sharing system.

The European General Data Protection Regulation (GDPR) was introduced in 2018 and modernised the legacy legislation of the 1995 Data Protection Directive (Directive 95/46/EC). This regulation mandates strict data protection obligations for all data, especially health. Healthcare organisations face a potential significant bottleneck in terms of compliance as the manual classification of vast amounts of unstructured clinical documentation is inherently prone to human error and compliance failure. According to IBM almost 50% of data breaches are caused by human error and IT failures. [4].

This project investigates the efficacy of semi-automating data a key element of governance and controls through Machine Learning (ML) assisted document classification. To support this a fully cloud-based, serverless application was developed on Amazon Web Services (AWS) to simulate a modern clinical environment which aligns with the “Digital For Care: a Digital Health Framework for Ireland 2024-2030” strategy which highlights ‘Data driven services’ and ‘Secure foundations’ as fundamental shifts for Irish healthcare as well as GDPR Article 25 [5] leveraging AWS technical development security features such as S3 managed encryption and IAM (identity access management). This also aligns with the NHS rollout of “A plan for Digital Health and Social Care”. The core of the research of this project is a comparative analysis of 3 ML implementations, being Logistic Regression using Sklearn, a fine-tuned BioBERT transformer model using Pytorch and the proprietary AWS Comprehend Medical Service.

To address the issue of class imbalance of clinical datasets which inherently omit personally identifiable information, this study incorporates the generation of synthetic data by leveraging locally installed Large Language Models (LLM) in line with [6] approach. These models were evaluated not only on classification accuracy specifically their ability to distinguish between Public, Internal and Restricted documentation, but also their computational latency, compute requirements and cost in a cloud-based deployment. The findings of this study offer a nuanced evaluation of the trade-offs between high compute transformer models and cost-effective classical approaches, providing a scalable and adaptable proof-of-concept approach for enhanced clinical compliance under GDPR Article 32 [7] which states the controller and the processor shall implement appropriate technical and organisational measures to ensure a level of security appropriate to the risk.

The results of this implementation indicate that ML-assisted document classification is a promising approach for supporting compliance in healthcare in terms of reliability, accuracy and cost. However, further research and domain specific applications such as real-world evaluation and model refinements would be required before such a system could be seen as commercially viable.

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Stephen Thompson

Stephen Thompson

Date: 19/04/26

Acknowledgements

Throughout the development of this project, I received overwhelming support and encouragement from a wide range of peers and loved ones.

I would like to take the time to specifically thank my supervisor Mr. Ciaran O'Driscoll for his unending support throughout this entire endeavour. Without this support and guidance this project would not have been possible.

I would also like to extend my deepest gratitude to my friends, family and loved ones for their love and support throughout this process.

Table of Contents

Abstract.....	ii
Declaration.....	iii
Acknowledgements.....	iv
1. Introduction	1
1.1. Project Background	1
1.1.1 Initial Concept & Project Evolution	1
1.1.2 Final Approach	2
1.2. Project Description	2
1.2.1 User Interface.....	2
1.2.2 Machine Learning Pipeline.....	2
1.2.3 Document Tagging, Security and Access Control.....	3
1.2.4 File Encryption and Storage	3
1.2.5 Audit Logging	3
1.2.6 Right to Erasure.....	3
1.2.7 Human Override – Human in the loop.....	3
1.3. Project Aims and Objectives.....	4
1.4. Project Scope	5
1.4.1 Scope Overview.....	5
1.4.2 User Interaction Scope.....	5
1.4.3 Machine Learning Scope	5
1.4.4 Document Generation Scope.....	5
1.4.5 Out of Scope.....	5
1.5. Thesis Roadmap.....	6
2. Literature Review	7
2.1. Introduction	7
2.2. Alternative Existing Solutions	8
2.2.1 Microsoft Purview	8
2.2.2 AWS Macie	9
2.2.3 Google Drive and OneDrive	10
2.2.4 Existing Solutions Conclusions	10
2.3. Technologies Researched	11
2.3.1 Cloud Providers.....	11
2.3.2 Data Generation.....	11
2.3.3 Machine Learning.....	12
2.3.4 Datasets	15

2.3.5 Docker Containerisation	15
2.3.6 PyTorch & Hugging Face.	16
2.3.7 Programming Languages, Database Solutions, Web Frameworks	16
2.4. Research for Architectural Decisions	17
2.4.1 Architectural Pivot	17
2.4.1 Consultation with GDPR Professional	18
2.5. Conclusions	18
3. System Analysis	19
3.1 System Overview.....	19
3.2 Requirements Gathering.....	19
3.2.1 Legislative Requirements	19
3.2.2 Existing Solutions and Literature Review	20
3.2.3 Technical and Machine Learning Research.....	20
3.2.4 Key Stakeholders.....	21
3.3 Requirements Analysis.....	22
3.3.1 Functional Requirements	22
3.3.2 Non-functional Requirements	23
3.4 Logical Architecture of System	24
3.5 Initial System Specification	25
3.6 Conclusions	26
4. System / Experiment Design	27
4.1 Introduction	27
4.2 Experimental Design	27
4.3 Software Methodology	27
4.3.1 Incremental Model.....	28
4.3.2 Application of Incremental Model	29
4.3.3 Design Pivot During Development	30
4.4 Project Planning	30
4.4.1 Project Schedule	30
4.5. Overview of System	31
4.5.1 System Overview.....	32
4.5.2 Machine Learning Layer Expanded	33
4.5.3 Fulfilment of System Requirements	34
4.5.3.1 Functional Requirements Fulfilment	35
4.5.3.2 Non-Functional Requirements Fulfilment	36
4.6 Conclusions	36

5. System / Experiment Development	37
5.1. Introduction	37
5.2. Software Development	38
5.2.1 Synthetic Data Generation	38
5.2.1.1 Phase 1 Pilot Data Generation	38
5.2.1.2 Phase 2 Public Data Generation	39
5.2.1.3 Phase 3 Restricted and Internal Data Generation	39
5.2.1.4 Phase 4 Testing Documentation Data for Deployment Testing	40
5.2.2 Sklearn	40
5.2.2.1 Pipeline Design	40
5.2.2.2 Sensitivity Mapping	40
5.2.2.3 Iterative Development	41
5.2.2.4 Deployment	41
5.2.3 BioBERT	42
5.2.4 Comprehend Medical	42
5.2.5 AWS Infrastructure	42
5.2.5.1 Cognito	43
5.2.5.2 AWS Managed File Service	43
5.2.5.3 DynamoDB	44
5.2.5.4 API Gateway	45
5.2.5.5 Elastic Container Registry	45
5.2.5.6 Lambda	46
5.2.6 User Interface	47
5.3 Conclusions	49
6. Testing and Evaluation	50
6.1. Introduction	50
6.2. System Testing	50
6.3. System Evaluation	51
6.3.1 ML Evaluation Methodology	51
6.3.2 Sklearn Evaluation	52
6.3.2.1 Sklearn Model 1.1	52
6.3.2.2 Sklearn Model 1.2	54
6.3.2.3 Sklearn Model 1.3	55
6.3.3 BioBERT Evaluation	57
6.4 Model Evaluation In Deployment	59
6.4.1 Results Evaluation and comparative analysis	61

6.4.2 Deployment Evaluation.....	66
6.4.3 Performance Metrics Summary	68
6.5 Results Evaluation and Comparative Analysis	69
6.5.1 GDPR Article 22	69
6.5.2 Cost Breakdown	70
6.5.3 Research Question Evaluation	70
6.6 Evaluate Project Management.....	72
6.6.1 Evaluate Project Plan	72
6.6.1 Evaluate Project Execution	72
6.7 Conclusions	73
7. Conclusions and Future Work.....	74
7.1. Introduction	74
7.2. Future Work	74
7.3. Conclusions	74
References	76
Appendix A: System Analysis	1
Appendix B: Prompts Used with Gen AI	3
B.1 Prompts Related to Coding	3
Appendix C: Additional Code Samples	4
Appendix D Testing Results:.....	5

1. Introduction

1.1. Project Background

The ongoing digitisation of modern society has fundamentally transformed how organisations generate store and manage information. Healthcare specifically represents a massively data driven sector that produces vast amounts of sensitive documentation daily across administrative, clinical and operational functionality. It is precisely for that reason that healthcare became the main focus for this system. The consequences of mishandling sensitive data at this scale make it a fitting and compelling case study for exploring semi-automated compliance.

Recent advancements in machine learning and generative AI have opened the doors for automated data classification in this environment [8]. The main motivation for this project was to question whether these recent innovations could be meaningfully applied to a clinical setting to reduce the manual compliance burden facing many organisations, and whether implementing this kind of system is not only more accurate but cost effective and efficient enough to be viable in a real clinical environment.

This presents the central research question for this project: Is machine learning a viable approach to access control in a clinical environment to reduce GDPR risks? This project aims to answer that question through a series of implementations and comparative analysis of 3 distinct machine learning approaches all applied to a fully serverless cloud architecture.

1.1.1 Initial Concept & Project Evolution

The original concept for this project as was written about in the interim report involved developing a locally installed app that would operate on individual machines within a clinical environment. During the development of the initial prototypes and further research post prototype numerous challenges were uncovered. The deployed system would require installation on all machines in a clinic with no reliable means for communication between machines. This opened the door to several compliance concerns when transporting documents between machines. For example, the system would be more than capable of processing information in its own environment but fundamentally there were no safeguards in place to prevent documents being intercepted or misused during transit such as over email or messaging. File sharing would have needed to be implemented externally which means governance becomes nearly impossible outside of the machine. Another issue was that of legacy systems and operating systems.

As of December 2020 46,000 HSE desktop machines still run Windows 7 which has seen no significant security updates since January 2020 [9] [10], there was no reliable basis to assume the software would have been viable on all machines in a clinic. This realisation as well as the HSE and NHS decision to pivot towards cloud-based architecture led to a natural architectural shift towards a cloud-based system using AWS. This decision strengthened the technical aspect of this system allowing for enterprise grade encryption across the entire system including AES256 at rest in the cloud and TLS1.2 in transit bringing this project up to enterprise standards.

1.1.2 Final Approach

This cloud based approach aligns the system in the direction of the HSE's Digital for Care: A Digital Health Framework for Ireland 2024-2030 [1] which presents a commitment to a full digitisation of Ireland's healthcare infrastructure with cloud-based solutions at the forefront of this commitment. This follows the NHS's active migration to cloud-based data solutions as part of its digital transformation program [3]. The effect of this deployment can be seen with Tallaght University Hospital in Dublin [11] showing real world deployment of AWS infrastructure for managing healthcare documentation .

By implementing this system using AWS cloud-based technologies the system reflects the standards that healthcare organisations are actively adopting, adding real world relevance to the system.

1.2. Project Description

This project was created for the purpose of safeguarding sensitive data produced by a clinical environment.

This is done by creating a fully cloud-native implementation of a file sharing platform which includes: a user interface for document upload, download, searching and deletion. An administrator interface consisting of immutable audit logs, classification reports and reclassification features.

This system will house the comparative piece of this report consisting of 3 distinct machine learning implementations, all designed to complete the same classification task of determining uploaded document sensitivities for role-based access. This underlying machine learning implementation forms the basis for comparative analysis for the project.

1.2.1 User Interface

A web-based GUI will be implemented using HTML, CSS and JavaScript. This remains unchanged from the interim project plan and will be the primary point of interaction for users. The interface supports document upload, storage, viewing and interaction.

A separate hidden administration dashboard has been implemented to allow for human override of documents. This allows for manual overriding of documentation. Each classification receives a tag on upload that is viewed with a confidence score on each upload to allow for a system administrator to view classification results at a glance and manually classify/ review any exception.

Authentication for access to the system is handled internally by AWS Cognito to support UI access using role-based user groups such as Doctor, Nurse, Admin, Facilities.

1.2.2 Machine Learning Pipeline

Three interchangeable machine learning implementations have been developed as the analytical base of this system. Each model performs the same task, of classifying into 3 separate categories of PUBLIC, INTERNAL, and RESTRICTED. These models are designed to be interchangeable during operation; each model is housed in a separate Lambda function that is triggered by upload to the S3 bucket which houses documentation. The exchanging of the models can be performed seamlessly without the system needing to be taken down by changing the S3 trigger event based off the desired model.

The 3 models employed for this system are:

- Logistic Regression classifier trained using sklearn
- a fine-tuned BERT/ BioBERT transformer model using PyTorch
- AWS Comprehend Medical

1.2.3 Document Tagging, Security and Access Control

Upon classification, each document receives a tag associated with a sensitivity level and is stored in a DynamoDB table alongside metadata including confidence score, time of upload, file size and classifying model.

Upon the request of a file by a user, the system extracts the user's role from the JWT token and compares to the documents assigned classification tag. Access is granted if the level of clearance is above that of the tag, and a pre-signed S3 URL with a 5-minute expiry allows for download of the document, this ensures no permanent access endpoints exist within the system which can be viewed as a security vulnerability.

1.2.4 File Encryption and Storage

Files are stored in a single scalable cloud-based S3 bucket, this solves the initial issue with the interim prototype of scalability. The size of these S3 buckets is infinitely scalable removing the issue of local document storage polluting local drives.

This S3 bucket handles the encryption for the system providing AES256 at rest and TLS1.2 in transit. These encryption protocols are seen as standards at industry level with AES256 and TLS being seen as the gold standard according to the National Institute of Standards and Technology (NIST) [12] [13]

Article 32 [7] of the GDPR requires organisations to implement appropriate technical measures to ensure security proportionate to risk, which has encryption explicitly cited as one such measure. The use of AES and TLS as mentioned above satisfies this requirement.

1.2.5 Audit Logging

Every interaction with the system is written to an immutable log trail contained in DynamoDB. This record of activity directly supports compliance with Article 30 [14] of the GDPR which states controllers must maintain records of processing activities. This also provides administrators with appropriate logs of all system actions should human intervention be required.

1.2.6 Right to Erasure

Each document in the system contains a deletion pipeline that permanently removes documents from S3 in line with Article 17 [15] of the GDPR which establishes the right to erasure of personal data upon request should such a request be deemed valid, as the right to deletion may be superseded by the need for retention for other legal purposes. A timestamped deletion entry is also written to DynamoDB to ensure actions are recorded.

1.2.7 Human Override – Human in the loop

It must be recognised that when working with confidence scores no result will be perfect. To ensure accuracy the system is equipped with a feature to manually override all decisions made by the acting model to ensure human in the loop compliance. Every override is recorded in the audit log in the event of malicious actors. This supports the core goal of this project being machine assisted governance not automated governance. This also aligns the project with the principle of Article 22

[16] of the GDPR which establishes the right that no decision should be made solely on automated processing.

1.3. Project Aims and Objectives

The central question of this project is: Is machine learning a viable approach to document classification in a clinical environment to reduce GDPR risks?

To address this question thoroughly and evaluate multiple possible implementations the primary aim of this project involves generating a fully cloud-based serverless document management platform, which will incorporate three distinct machine-learning classification pipelines to determine the most accurate, cost-effective and clinically viable approach to semi-automated GDPR compliance in a healthcare environment.

Cloud Infrastructure Objectives:

- Design and deploy a fully cloud-based backend architecture using AWS
- Implement cloud-based document storage and immutable audit logs with industry standard encryption protocols

Machine learning objectives:

- Design and deploy three distinct machine-learning classification pipelines:
 - A classical Logistic Regression implementation
 - A fine-tuned BioBERT transformer model
 - A proprietary AWS Comprehend Medical based model
- Generate synthetic data locally through multiple design iterations to enrich and enhance datasets as well generate unseen documentation for deployment testing
- Train and evaluate each model across multiple iterations using standardised metrics influenced by peer reviewed academic literature.
- Perform a cost analysis of each final implementation both per inference and at scale to extrapolate possible costs of each implementation

Evaluation Objectives:

- Conduct a comparative analysis of each final implementation of accuracy, computational cost, inference latency, implementation cost and GDPR explainability
- Critically evaluate the relationships and trade-offs between model complexity, accuracy and cost
- Evaluate the limitations of the full system and individual implementations to encapsulate total research and potential commercial viability

Compliance Objectives:

- Implement role-based access control aligned with GDPR best practices of data minimisation
- Implement features and strategies aligned with GDPR best practices of data locality and immutable logging.

1.4. Project Scope

1.4.1 Scope Overview

The scope of the project is a cloud-based, GDPR-compliant document management and file sharing system for a simulated hospital environment. In its core functionality the system is comparable with proprietary file sharing platforms such as Microsoft One Drive and Google Drive, however its unique selling point lies in the integration of machine learning assisted compliance directly into the file management pipeline. Rather than treating compliance as an administrative burden/ afterthought, the system embeds compliance into its foundations in line with Article 25 [5] of the GDPR which mandates privacy by design and default.

The system provides a centralised document repository through which authorised clinical staff can upload, access and manage files. This centralised approach addresses a key challenge in clinical environments, the secure and efficient distribution of documentation between departments and individuals without risk of accidental exposure to unauthorised actors. This is directly aligned with the GDPR principle of data minimisation outlined in Article 5(1)(c) [17], which requires that personal data be accessible only to those with a legitimate need to access it.

1.4.2 User Interaction Scope

Users interact with the system entirely through a web browser requiring no software installation. Accounts are managed and distributed rather than providing registration functionality. AWS Cognito assigns a role into one of 4 categories being: Doctor, Nurse, Admin and Facilities, each with a corresponding level of document access level determined by the classification of each file. This role-based approach ensures decisions are driven by the determined sensitivity of the document rather than by manual administrative process

1.4.3 Machine Learning Scope

This foundation provides the platform for which the 3 separate machine learning implementations act upon. These 3 implementations with multiple iterations of each provide the analytical base for comparative analysis and evaluation discussed in Chapter 6.

1.4.4 Document Generation Scope

The machine learning research is supplemented by the further implementation of a document generation pipeline. This pipeline enriches the datasets in which the ML implementations are acting upon, providing depth and addressing imbalance discussed further in Chapter 4.

This system is a proof-of-concept designed to evaluate the viability of ML-assisted document classification in a simulated clinical environment with a comparative analysis of three separate machine-learning approaches contained within functionality.

1.4.5 Out of Scope

This project is not intended to be a production ready enterprise deployment. It does not address network-level security concerns beyond AWS defaults, real world patient onboarding, does not claim full GDPR compliance across all articles, does not claim to advance state of the art research in natural language processing.

1.5. Thesis Roadmap

The following is a summary of the subsequent chapters of this thesis.

- Chapter 2
 - Literature review into existing solutions, and similar research to the project. This chapter will outline the research that led to the creation of the system and how it influenced design decisions.
- Chapter 3
 - This chapter will discuss overall system design decisions and implementations including considered methodologies and project evolutions over time.
- Chapter 4
 - This chapter will discuss the decisions made to develop the system including model creation and training. Synthetic data generation and the backend system creation.
- Chapter 5
 - This chapter will showcase the steps taken during development, specifically the prototyping and incremental designs required to ensure the models and thus the systems were designed with the utmost scrutiny to ensure unbiased and fair results. This chapter will also discuss the implementation of the cloud based backend and document generation pipeline as well as the challenges and decisions required to create a fully fleshed out system.
- Chapter 6
 - This chapter will discuss the testing and evaluation strategy for this project as well as the results yielded by the machine learning models training and evaluation. A cost analysis is also conducted to ensure a thorough comparative analysis.
- Chapter 7
 - This chapter will discuss possible future work and research that this project creates as well as the results of testing and final evaluations of the multiple systems created for the comparative analysis.

2. Literature Review

2.1. Introduction

This chapter provides an overview of existing literature and solutions relevant to the scope of the project. This includes existing projects and literature that inspired the solution, proposed and implemented technologies as well as the design decisions and literature that led to their adoption. This chapter will also examine the legislative framework associated with this project specifically discussing the relevant articles from the General Data Protection Regulation (GDPR) [18].

This chapter will also discuss the breadth of literature that decided the approach for synthetic data generation for model implementation. This included research into the viability of data generation for specifically personally identifiable information.

Alternative cloud-based hosting systems will also be examined and the choice of Amazon Web Services (AWS) over these solutions will be discussed.

The choice to pivot from traditional SQL (Standard Query Language) based blob storage to a dual pronged blob repository and a document-based audit log will be discussed.

The choice of which front end framework to be used as well as possible alternative solutions will be discussed in depth.

Crucially the research for model choice will be scrutinised as this is the core research question of the thesis.

2.2. Alternative Existing Solutions

2.2.1 Microsoft Purview



Microsoft Purview is a comprehensive data governance platform operating within the Microsoft 365 ecosystem, providing automated document classification, activity monitoring, immutable logging and risk management.

Purview represents the greatest inspiration and closest existing enterprise solution to the system proposed in this project. Purview is a comprehensive data governance tool that operates on the Microsoft ecosystem which provides automated document classification, activity monitoring and immutable logging and risk management features. Similar to the system of this project Purview also classifies documents interacted with across Microsoft 365 services, all interactions are logged and monitored, and suspicious activities can be flagged and reviewed. This architectural structure directly influenced the direction of this project.

Implementation Limitations

However, Purview presents several implementation limitations when considered in the context of a small to medium sized business/ clinic. The platform offers broad ranges of products and services, [19] Including but not limited to data loss prevention, data in transit protection, insider risk management and compliance reporting. While the breadth of functionality available cannot be overstated and would be seen as a massive strength for large scale institutions looking to remain compliant and competitive it simultaneously represents a barrier for adoption to smaller organisations. A small clinical practice with fewer than five staff members cannot reasonably be expected to deploy and maintain a compliance tool of this complexity. This market encapsulates the majority of clinical settings in Ireland [20] there are roughly 2500 general practitioners practices in Ireland as of 2022 as opposed to 86 hospitals according to the National Healthcare Statistics as of 2024. [21]

This complexity also poses another issue of incorrect use leading to data leaks and malpractice as it cannot be expected of a small clinic to understand the intricacies of such a tool as well as compliance law this is shown as mentioned in the Abstract that According to IBM almost 50% of data breaches are caused by human error and IT failures. [4]. This complexity can also lead to a false sense of security when in reality the clinic may be just as exposed.

Cost Implication

Cost presents a massive hurdle as Purview is part of E3 and E5 licensing tiers which may fall out of budget for many small organisations. [22]. Purviews advanced compliance features require Microsoft 365 E5 licensing priced at €38.40 per user per month in Ireland [23] or €10.40 per user per month for standalone Purview [24]. To extrapolate these numbers, St Vincents Hospital Group employs 5,298 staff as of 2024 according to their annual report [25]. This would equate to €55,099 per month or €661,190 per year for standalone Purview assuming all staff required access to Microsoft Purview. Microsoft further restricted access by removing Business Premium as an eligible prerequisite for the Purview suite.

Proposed Solution

The system proposed by this project addresses these limitations directly. Rather than presenting compliance as a complex, out of reach concept the project embeds data governance principles in its design, this technology is invisible to end users and aims to be a plug and play design that could be applicable to any level of deployment.

2.2.2 AWS Macie



AWS Macie represents the closest architectural parallel to the machine learning functionality of this project. Macie is a fully managed AWS service that leverages machine learning and pattern matching to detect, discover and classify documents to protect data in S3 buckets for AWS projects. Macie automatically detects and classifies a broad range of sensitive data types including PII, financial data, health data and alerts when it believes sensitive data has been misclassified. From this description it is clear to see the direct parallels between Macie and the machine learning aspect of this project and could be seen as a direct enterprise grade solution of the classification pipeline.

Macie is customisable allowing institutions to add their own sensitivity tuning tailoring each instance of Macie for specific tasks relevant to its environment.

Cost Implication

From a cost perspective Macie is consumption based, charging [26]:

- \$0.1 per S3 bucket per month
- \$0.01 per 100,000 objects monitored
- \$1 per GB of data inspected.

Macie also provides immutable logs and compliance reporting proposed by the project, but this is displayed in the AWS console only visible to those with access and the knowledge of AWS. The system addresses this limitation with a layer of abstraction between end users and AWS. Clinical staff only interact with the simple web interface described in Chapter 5.

As will be demonstrated in the evaluation chapter Macie's consumption-based pricing model will be pitted against the projects own implementations to add nuance to the comparative analysis of the system.

Proposed Solution

It should be noted that direct cost comparison between Macie and this projects implementation is not straightforward. Macie's consumption-based price model may prove to be more cost effective than a containerised fine-tuned BioBERT classifier, but this does not take into account the transparency and auditability of the project's solutions. Similar to Comprehend Medical Macie is a black box for classification, the underlying model, training data and methodology are unknown at the time of writing. This could contradict Article 22 of the GDPR [16] stating the basis for classification must be demonstratable and explainable. The implementation in this project meets this criteria as all details of classification are documented

2.2.3 Google Drive and OneDrive



Google Drive & Microsoft One Drive present the gold standard solution for cloud storage in industry and personal use. Both platforms provide intuitive file management solutions and collaborative frameworks within their respective solutions; these solutions also provide deep integration with other products within their respective ecosystem such as Gmail and Outlook. It is precisely this integration that limits the context of clinical deployment. Organisations adopting either architecture become inherently locked into the surrounding ecosystem creating dependency risks which is detrimental to a healthcare environment where document access is critical.

More fundamentally although both solutions offer world class security from external intervention neither was designed with automated compliance in mind. The responsibility for access control and governance falls entirely on users and system administrators. Document sharing is manual; permissions are manually configured and access manually revoked. According to the Hospital In-Patient Enquiry (HIPE) 1.7 million hospital discharge records are created annually [27], using this statistic we get a sense of the scale needed for documentation in Ireland. This introduces a significant operational overhead in terms of data protection, coupled with the Digital for Care initiative [1] this is inherently prone to human error. Furthermore, neither platform offers automated sensitivity classification or ease of access to immutable logging which supports compliance according to GDPR Article 30 [14].

Despite these limitations, both platforms provide key insight into the gold standard for upload, download and browsing experiences of interacting with a cloud storage solution and provide fundamental inspiration for the overall UI/UX design of the system.

2.2.4 Existing Solutions Conclusions

Across the existing solutions examined a clear pattern emerges. Consumer platforms such as Google Drive and One Drive prioritise usability but lack automated and rigorous compliance tools. Enterprise solutions such as Purview offer comprehensive governance but at the cost of complexity that excludes small to medium sized clinical practices in Ireland. Cloud based ML services such as AWS Macie do provide powerful and reliable automation but the lack of transparency and fluctuation in cost provides no guarantees. The system proposed through this research occupies a distinct niche, combining automated ML classification, role-based access control, immutable audit logging and GDPR compliance by design in a system designed to be accessible to clinical environments.

2.3. Technologies Researched

2.3.1 Cloud Providers

Three cloud providers were considered for the system. Through researching for this project AWS, Microsoft Azure and GCP were identified.

The decision for AWS was determined based off market share, industry relevance and data locality.

Below details the criteria for cloud providers to determine the architecture choice for this project.

Market Share

AWS shows the highest market share as of Q4 2025.

According to Statista as of Feb 2026 AWS still holds its dominant market share accounting for 28% market share compared to 21% for Azure and 14% for Google Cloud in Q4 2025.

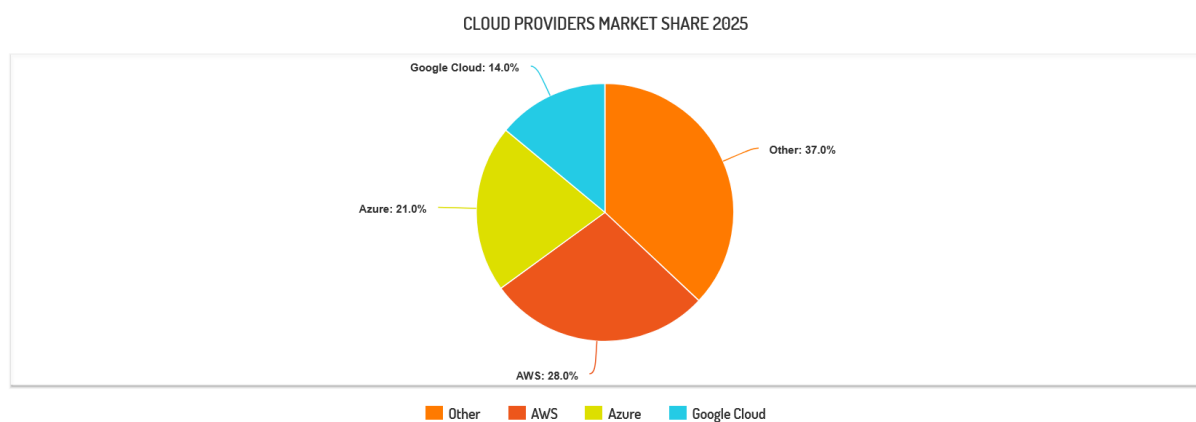


Fig2.1 Market Share pie chart

Industry Relevance

AWS industry relevance extends beyond clinical implementations.

AWS is also detailed in the NHS plan for digital health and social alongside Microsoft Azure.

Although Microsoft Azure is the main operator in the NHS digital health plan the use of AWS avoids the lock in problem discussed with Purview and creates architectural independence.

Data Locality

From the viewpoint of GDPR compliance AWS eu-west-1 server is based in Dublin making the argument for data locality which demonstrates compliance in line with Article 44 [28] of the GDPR regarding cross border control lean towards AWS. It should be noted that the GDPR allows for data transfer across borders/ jurisdictions under warranted circumstances, but this implementation removes the need for such an assessment

2.3.2 Data Generation

Synthetic Data

A large portion of this project relies on synthetic data generation to supplement the chosen dataset particularly for the internal and public sensitivity classes which are underrepresented in real world examples due to the sensitive nature of personally identifiable information and the lack of clinical

relevance to non-medical documentation. The original hypothesis for using synthetic data generation stemmed from the need to train the models on personally identifiable information if the system was ever to gain real world relevance.

Synthea

The concept of synthetic data generation is an established field in health informatics research. Tools such as Synthea from Synthetic Health were considered for the implementation.

Synthea, an open-source synthetic patient generator developed by the MITRE Corporation [29] demonstrates a viability of generating research-grade anonymised medical documentation at scale. Synthea has been used and cited in multiple studies such as Chen et al. [30] as well as the original Synthea study [31]. These studies document Syntheas validity for generating patient documentation at scale but due to Syntheas limited ability to generate personally identifiable information such as phone numbers, email addresses etc and the use of an API-accessible model which would violate the data locality principle of the project. Synthea was ultimately not chosen for this implementation.

Local Large Language Models

This led to the researching of locally hosted large language model approach for synthetic data generation. The approach of using LLM's for synthetic data generation has seen success in studies such as [32] which states the synthetic NLP model trained on LLM generated text outperformed models trained on real clinical text and physicians were unable to distinguish real from synthetic in a Turing test, and [6] which states a fine-tuned llama 3.2 model hosted locally using Ollama containerisation could reliably generate convincing medical documentation for NLP research.

Finally, the use of OpenAI's GPT API [33] was considered but explicitly ruled out due to the concerns of data locality when generating information as stated in the Synthea justification.

The final decision for data generation fell to using a local instance of llama 3.2 via Ollama. This approach satisfies both the research requirement for realistic synthetic data and the GDPR data locality requirement ensuring that no data was transmitted to external servers outside of the scope of the project.

2.3.3 Machine Learning

The choice of a 3-model comparison came from the intention to provide a distinct differentiation of approaches for the comparative analysis task. A classical implementation to establish a transparent and computationally inexpensive baseline, a pretrained transformer model to represent state of the art domain specific NLP. Finally, a proprietary solution to represent blackbox commercial architecture. This 3-tiered approach was deliberate to produce a comparative analysis on a broad range of implementations to ensure experimental justification and providing the range of results necessary to draw a meaningful conclusion about model viability in a clinical deployment context.

Classical Implementation: Scikit-learn

The classical machine learning baseline for this project was implemented using scikit-learn, which is a widely used open-source Python library that is used as a foundation in machine learning education and practice. Sklearn's extensive documentation, peer reviewed research [34], community and open-source nature made it a leading competitor for use as a baseline implementation in the context of comparative research.

The transparent and reproducible nature of Sklearn is well documented in peer reviewed literature with widely cited papers on NLP classification [35] , the focus of this project.

Three classical algorithms were implemented and evaluated for the baseline model of the system: Logistic Regression, Linear Support Vector Machine, Random Forest. Each algorithm was trained against an identical dataset with consistent parameters to ensure fair comparison. The results of this analysis are presented in full in Chapter 6.

Sklearn Algorithms

Random Forest: Uses many decision trees to make predictions. Each tree scans differing sections of data and their results combined by voting for classification or averaging for regression. Each tree selects random features to decide how to split data this ensures each tree differs from each other. For classification each tree gives its own answer, and the final answer is concluded through majority voting

Logistic Regression: Models relationships between predictor variables and the logarithmic odds of events, using a sigmoid function to map a probabilistic outcome, this is adapted for the project by making this binary outcome for each level of classification and returning the highest result.

Linear SVM (Support Vector Machines): A type of supervised machine learning algorithm used for classification and regression. Widely used in pattern recognition, image analysis and NLP. Linear SVM operates by finding the optimal hyperplane that separates data points into different classes.

Logistic Regression received the highest score across the board during this testing period making it the logical choice for the baseline model implementation of the project.

Pretrained Transformer Model: BioBERT

Bidirectional Encoder Representations from Transformers (BERT) [36] represents a leap forward in transformer models. Unlike previous transformer models, BERT processes text by considering both left and right context simultaneously enabling deeper understanding of linguistic structure and semantic meaning.

This bidirectional context is particularly important for tasks such as document classification where meaning is often dependent on surrounding terminology.

Additionally, BERTS large training corpus allows for large general language representations that can be fine-tuned for individual task representation. This makes BERT highly adaptable and efficient where labelled data is limited.

Clinical BERT	Clinical BERT is a fine-tuned BERT model developed by [37]and further trained and fine-tuned on clinical notes and discharge summaries, although clinically excellent the lack of general medical documentation makes it unsuitable for this task
BioBERT	BioBERT represents a ‘happy medium’ for the task of the project. BioBERT being trained on PubMed abstracts and PMCtexts represents a much broader medical view as well as keeping the PII detection found in general BERT ensures a wide variety of training data while still being fine-tuned enough to support the task of the project.
RoBERTa	RoBERTa [38] is an improved BERT model trained on significantly more data 160gb using longer batches and removing Next Sentence Prediction. Although refined and generally excellent it lacks the pre tuning to medical terminology required for this task

Proprietary BlackBox Model: AWS Comprehend Medical

AWS Comprehend Medical is a HIPAA-eligible, natural language processing service provided by AWS that uses machine learning to process structured and unstructured information without the need for prior training [39].

This will be used as the highest level pretrained model in the system as it presents an untuned Blackbox.

Comprehend Medical has functionalities such as DetectEntitiesV2 available through its API that detects medical conditions, dosages, anatomy etc, and DetectPHI which extracts PII like names, addresses, birth dates used to manage data privacy.

Amazon Comprehend is another available resource similar to Comprehend Medical. Comprehend works in the same fashion as comprehend medical but without the fine tuning making it unsuitable for this project.

2.3.4 Datasets

MTSamples

MTSamples [40] is a publicly available collection of approximately 5,000 anonymised medical transcriptions spanning over 40 clinical specialties. MTSamples was selected as the initial dataset for training and fine tuning of Logistic Regression and BioBERT respectively. MTSamples represents a widely used open-source anonymised dataset. [41] & [42] recommends the use of MTSamples for NLP citing the 40+ specialties and structured nature of the dataset. [42] specifically provides results and states valid results using Logistic Regression for clinical text validation.

PUBMed

PUBMed [43] was considered to supplement the dataset for training and fine-tuning. As a repository with a vast corpus of domain specific text the choice seemed appropriate. However, MTSamples was deemed more suitable for the transcription style classification task provided without API integration.

European Health Data Space

The European Health Data Space initiative [44] was identified as a potential source of clinical datasets. The data space provides researchers with a vast repository of EU clinical records however at the time of dataset selection the platform's available datasets were primarily structured records rather than clinical transcriptions with free-text datasets requiring institutional approval beyond the scope of the project.

2.3.5 Docker Containerisation

Docker [45] is an open-source containerisation platform that allows applications to be packaged in isolated containers that consistently run across different environments. This containerisation technology is compatible with AWS ensuring containers will run correctly in a cloud environment. Docker retains near-universal adoption in modern software development, reaching 71% adoption among professionals in 2025. Docker is also used by over 20 million developers worldwide. In the context of this project Docker was required based on the essential need for sklearn and PyTorch or Hugging Face. By containerising and packaging these models into a Docker container the full ML pipeline could be deployed to AWS without dependency conflict.

2.3.6 PyTorch & Hugging Face

PyTorch [46]. is an open-source deep learning framework developed by Meta's AI research lab.

Pytorch is widely regarded as the leading framework for academic and research-oriented deep learning, valued superficially for its dynamic computation graph, intuitive Python first design and strong GPU acceleration.

PyTorch was selected as the framework to host BioBERT due to its native support for model fine-tuning and seamless integration with the Hugging Face Transformers library.

Hugging Face Transformers was selected to implement the BioBERT model due to its extensive support for pretrained transformer architectures and seamless integration with PyTorch. This significantly reduced development complexity and enables efficient fine-tuning and deployment of the model. Additionally, Hugging Face promotes reproducibility through standardised model configurations and versioning which aligns with the experimental nature of this project

2.3.7 Programming Languages, Database Solutions, Web Frameworks

Python

Python [47] is widely recognised as the standard programming language for machine learning and natural language processing

Python's high level; concise syntax allows for streamlined development as well as provides a vast toolkit relevant to machine learning and natural language processing. Pytorch, NumPy and pandas are python specific libraries determined pivotal to the development of this project.

Python is widely supported in the AWS toolkit in Lambda making it a logical choice for machine learning in AWS. AWS provides SDKs for Python allowing for streamlined integration with the serverless backend [48].

SQL – DynamoDB

In choosing a database framework for this project initially it seemed logical to choose a lightweight relational database for document storage. Document based non-relational databases such as CouchDB seemed unsuitable for the project due to the perceived need for efficient joins, aggregations and retrieval that were pivotal to system efficiency. Therefore, an SQL based database specifically MySQL was the logical decision for storage. This was highly compatible with the previous stack used in prototype development.

During the architectural pivot of the project DynamoDB was chosen as the logical architecture choice as an AWS service for databases.

This complements the S3 bucket for storage by providing document tags without the need to store blob objects, which has been passed to S3. This implementation also supports streamlined development due to the lack of connection management required, automatic scaling and the removal of the local storage issue as well as seamless integration into the AWS stack.

This pivot is discussed further in Chapter 4

HTML, CSS, JavaScript

HTML, CSS and JavaScript remain the standard technologies for web-based applications, due to the platform-independent and flexible nature of these standards maintainable, accessible and iterative development of the UI is possible which in turn streamlines development.

Due to the user interfaces simple nature of two a two-page structure it was decided React, Vue and Angular would have added unnecessary bloat and compatibility issues with the serverless backend.

2.4. Research for Architectural Decisions

2.4.1 Architectural Pivot

The proposed system underwent a significant architectural pivot from the design outlined in the interim report. This prototype implementation developed as part of the interim system proposed a locally deployed system consisting of a flask web server, MySQL relational database for document classification and storage, and locally executed python encryption scripts with different tiers of encryption for data protection. While this architecture provided a sufficient proof of concept during further development and research it was discovered that this approach reached fundamental limitations when attempting to apply the logical architecture to real systems these being; functionality, GDPR compliance, cost, modern healthcare infrastructure and security.

New AWS Approach

The pivot to fully serverless AWS architecture resolved these limitations simultaneously. S3 provided infinitely scalable centralised document repository accessible from any authenticated browser removing both the local machine memory constraint and the lack of file sharing functionality. This approach aligns the system with modern enterprise cloud solutions such as Google drive discussed in section 2.2.3. Lambda functions handle all backend logic and are triggered only when required as discussed in Chapter 5 which removes the need for a perpetually operational application server which removes the cost and maintenance overhead. API gateway provides HTTPS endpoints for all client server communications.

GDPR Compliance

The GDPR implications of the original system also presented risk. A locally hosted MySQL database storing sensitive clinical documentation would require an organisation to manage its own security infrastructure such as encryption at rest and in transit, access control, and maintaining backups. For a small organisation this can represent an unwanted or unaffordable cost.

The AWS architecture addresses these GDPR concerns by design. All documents stored in S3 are encrypted at rest and in transit by default. Data locality is still guaranteed by deploying the system in AWS region eu-west-1. CloudWatch provides an additional layer of immutable logging independently of application code. Accompanying this with system implemented DynamoDB logging ensures accountability by design which the original implementation could not provide.

Cost

The cost implications of both architectures differ fundamentally. A locally hosted system requires a significant upfront cost in hardware, maintenance and security. These costs are incurred regardless of system use. A system with zero active users incurs the same cost as one under peak load. For smaller clinical organisations this approach is neither scalable nor financially viable.

The serverless AWS architecture flips this model entirely. Charges are only incurred during execution, as discussed in Chapter 6. Cost metrics are measurable to the cent with S3 charging per GB of storage and DynamoDB charges per read and write meaning organisations are only paying for what they use not potential future use.

Alignment with Modern Healthcare Infrastructure

Both the HSE's Digital for Care 2024-2030 framework [1] and the NHS's Digital for Health and Social Care [3] identify cloud-based infrastructure as the direction of travel for healthcare IT.

A locally hosted system while sufficient for proof of concept does not align with modern health infrastructure trends. By using cloud-based, serverless design principles the project demonstrates an architecture that could be implemented in modern solutions.

2.4.1 Consultation with GDPR Professional

In addition to the literature review highlighted above regular informal consultations and discussions were conducted with the Executive Director of Privacy, Data Governance, and AI at a large international organisation who has over 25 years of experience working in data privacy and security. These discussions encompassed system design in line with GDPR requirements for document handling, system architecture and interpretation of GDPR legislation. Although informal these consultations provided real world insight and context to the foundations of the academic research.

2.5. Conclusions

This chapter has provided a comprehensive overview of existing solutions, technologies and research that has informed the design and implementation of this project. The examination of alternative solutions presents a gap in the market for a unique approach combining semi-automated ML classification, role-based access control, immutable audit logging and GDPR compliance by design in a system accessible to clinical environments at any scale.

The technology decisions documented in this chapter were each justified with reference to functionality, compliance and cost. The architectural pivot from local deployment to fully cloud-based backend functionality was demonstrated to be a necessary evolution that strengthened the system in functionality, security, compliance and cost.

The research from this chapter directly informs the subsequent decisions in later chapters.

3. System Analysis

3.1 System Overview

The system from a user standpoint functions similar to existing software solutions such as Google Drive, Microsoft One Drive and iCloud, with a key difference. The system is not designed for the everyday user to upload store and share files; the goal is efficient clear documentation sectioned not by personal partitions but instead to act as a repository for files that need to be accessed at a moment's notice. Due to the breakneck pace of clinical environments, the repository acts as a system overview for actors within the organisation rather than having needed to provide and remove access the system semi-automatically provides access to those who require it ensuring ease of access to all that require it.

3.2 Requirements Gathering

The requirements gathering methodology for this project was deliberately legislation first in approach as it was imperative that this system was GDPR compliant. The GDPR was used as a primary source which shaped design decisions throughout the project. This was supplemented by a review of the technologies discussed in Chapter 2.2 & 2.3 as well as peer reviewed literature and consultation with an industry data protection professional.

This approach produced a four-part structure to establish baseline requirements for the system.

Part 1: Legislative Research

Identify the GDPR articles relevant to the system and translate each into a concrete system requirement

Part 2: Existing Solutions and Literature Review

Examine existing enterprise implementations, cloud implementations and NLP research to determine what features a viable solution must provide and what approaches are validated

Part 3: Technical and Machine Learning Research

Examine technical standards, architectural options and ML evaluation methodologies to inform non-functional requirements and experimental design

Part 4: Stakeholder Analysis

Identify key actors affected by such a system and ensure their needs are reflected in implementation

3.2.1 Legislative Requirements

The technical specification was explicitly drawn from interpretations of GDPR legislation and consultation with an industry professional. The GDPR was used as a primary source which shaped design decisions throughout development. As discussed in Chapter 1 the key articles informing requirements were:

- Article 5 (data minimisation)
- Article 17 (right to erasure/ right to be forgotten)
- Article 22 (human oversight for automated decisions)
- Article 25 (data protection by design)
- Article 30 (records of processing activities)
- Article 32 (security of processing)
- Article 44 (data locality). Each of these articles directly influenced the functional and non-functional requirements detailed in 3.3.1 and 3.3.2.

3.2.2 Existing Solutions and Literature Review

This part of requirements gathering produced FR1, FR10, FR11 & FR12. The examination of existing solutions and peer reviewed literature conducted in Chapter 2 informed several functional requirements of the system. Google Drive and OneDrive established baseline expectations of a cloud-based file management service. The ability to upload, browse search, filter and download documents is a standard user experience across both platforms. Examples of this can be seen in figure 3.1 below.

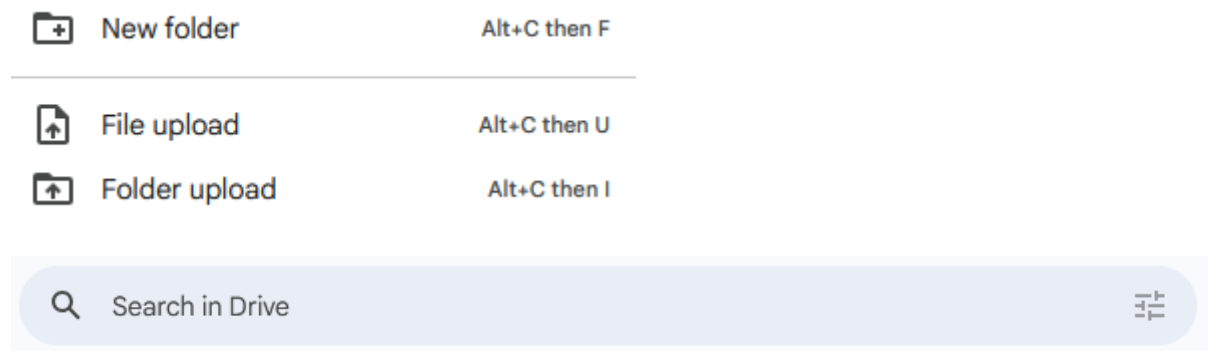


Fig 3.1 Google Drive Search & Upload

Microsoft Purviews architecture demonstrates human oversight and immutable audit logging at an enterprise scale, combined with Article 30 and Article 22 discussed in 3.2.1 formed the basis of FR9 and FR6.

AWS Macie demonstrated automatic sensitivity classification and role-based access control directly contributing to the foundations of FR2 & FR3.

3.2.3 Technical and Machine Learning Research

Technical research discussed in Chapter 2 informed the non-functional requirements of the system. The AWS Well-Architected Reliability Pillar [49] influenced NFR4.

The NIST standards for AES256 and TLS influenced NFR2 which establishes specific encryption standards are required to satisfy Article 32 of the GDPR.

Nielsens response time heuristic [50] established the baseline for NFR3, the principle that system responses completing in less than one second allow a user to maintain their train of thought, this also influenced the design of FR12 opting for an automatic filter to searching rather than waiting for results.

This also influenced the decision to ensure system calls were made asynchronously to avoid longer response times on larger models or model cold starts.

ML Metrics

The Machine Learning evaluation methodology using metrics such as precision, f1, recall and accuracy scores as well as standardised train/test splits was adopted in accordance with Naidu et al. [51] and Gholamy et al. [52] which establish these metrics as appropriate evaluation methodologies and metrics for multi-class classification evaluation.

3.2.4 Key Stakeholders

Clinical Staff

Clinical staff will be the primary end users of the system across all levels of a hospital/ clinical environment. Doctors, nurses, administrative personnel and facilities staff. Each group will interact with the system differently with a differing level of clearance which directly influences the implementation of Cognito roles in Chapter 5, all documents will be visible to every tier of user but those without clearance will be unable to modify, move or delete documents. The system must remain intuitive enough to not impose an additional burden on staff who typically are under time pressure in a fast-paced environment.

Data Protection Officers

Data protection officers present another key stakeholder of the system ensuring organisational compliance with the GDPR. For this stakeholder group the system potentially provides massive value. This system aims to reduce the burden of manual compliance; every classification result is logged and visible in an immutable audit trail on a protected endpoint only accessible to those with appropriate clearance.

The human override feature also provides additional safety rails to ensure compliance, DPOs can view audit logs and override a compliance decision within seconds of the classification taking place which aids the system in remaining true to human-in-the-loop governance pertaining to Article 5(2) [17] of the GDPR.

System Administrators

System administrators represent the operational stakeholders responsible for managing day-to-day operation and oversight of the system. This would include providing accounts to clinical staff from AWS Cognito, monitoring the classification dashboard and performing reclassifications should the model produce an inaccurate result. The human override feature provides administrators with direct mechanism for correcting misclassifications with every override written to the logs.

Patients

While patients will not interact with the system directly it is patient data that is primarily being processed by the system, be that medical records or personal information such as phone numbers and addresses. The system exists primarily to protect them and their data ensuring their sensitive records are only accessible to those with explicit clearance to view them. This is in line with Article 5(1) [17] of the GDPR.

3.3 Requirements Analysis

Through the requirements gathering from Chapter 3.2 the following functional and non-functional requirements were determined for the system.

3.3.1 Functional Requirements

This table details the non-functional requirements gathered from Chapter 3.2 and denotes requirement number, name and description.

FR1	Document Upload	The system must allow for authenticated users to upload documents of reasonable expected type through the web interface
FR2	Automatic Classification	Upon upload the system is expected to automatically provide classification for the document into determined sensitivity tiers. RESTRICTED, INTERNAL, PUBLIC or QUARANTINE based off the machine learning model in uses highest confidence of classification.
FR3	Access Control	The system shall automatically restrict document access based off the determined level of sensitivity meaning no access shall be granted to users without relevant clearance to view, download or generally interact with the document
FR4	Encryption at Rest	The system shall comply with enterprise grade security standards for information at rest, that being AES256 encryption.
FR5	Encryption in Transit	The system shall comply with enterprise grade security standards for information in transit, that being TLS encryption.
FR6	Immutable Audit Logging	To comply with GDPR Article 30 [14]the system shall write immutable logs to the document-based database for all system interactions including, upload, classification, access attempts, deletion, overrides and downloads.
FR7	User Authentication	The system shall support user authentication through provided accounts, due to the nature of deployment account creation will not be provided, each provided account shall be given a role indicative of the user's job title and access level.
FR8	Right to Erasure	The system shall comply with GDPR Article 17 [15] allowing the right to erasure for any individual that requests document deletion.
FR9	Human Override	In the event of a misclassification by the system human intervention will allow for reclassification allowing for human-in-the-loop compliance to ensure strict adherence to data protection legislation
FR10	Document Listing	The system shall present all available documentation pertaining to level of access of the user
FR11	Document Download	The system shall allow users with relevant clearance to download documentation and write actions to audit log
FR12	Document Searching	The system shall allow users to search through documentation using a search bar.

3.3.2 Non-functional Requirements

This table details the non-functional requirements gathered from Chapter 3.2 and denotes requirement number, name and description.

NFR1	Scalability	To ensure alignment with enterprise grade cloud storage and file sharing solutions discussed in Chapter 2 the system shall be infinitely scalable allowing for large volumes of documents to be ingested into the system without issue.
NFR2	Security	To ensure alignment with enterprise grade cloud storage and file sharing solutions discussed in Chapter 2 the system shall secure all documents with enterprise grade security consistent with the requirements of Article 32 of the GDPR [7].
NFR3	Performance	The system shall complete tasks such as uploading, classifying, and updating of documentation at speeds comparable to enterprise grade solutions
NFR4	Reliability	The system shall be available on demand without scheduled downtime, leveraging the serverless architecture to ensure uninterrupted availability.
NFR5	Compliance	The system shall to the best possible standard comply with the requirements of the GDPR throughout all stages of operation.
NFR6	Usability	The system shall be accessible via standard web browser without requiring local software installation or custom hardware configuration
NFR7	Data locality	In line with Article 44 of the GDPR [28] all infrastructure and data shall remain local to the operating region of the system (Deployed on AWS server eu-west-1 Dublin)

3.4 Logical Architecture of System

The results of the requirements gathering and requirements analysis produced the logical architecture of the system, the architecture adopted was a layered architecture split as seen in figure 3.2. The presentation layer corresponds to the user-facing browser interface through which all staff and administrators will interact. The business logic layer corresponds to the core application functionality. The Data storage layer represents the persistent storage of documents and database tables for classification records and audit logs.

All interactions flow sequentially through each layer, ensuring no users can interact with data or database records without first passing through authentication and access-control mechanisms of the business logic layer.

This diagram represents a high-level overview of system interactions. The physical implementation of each layer is discussed in Chapter 4.

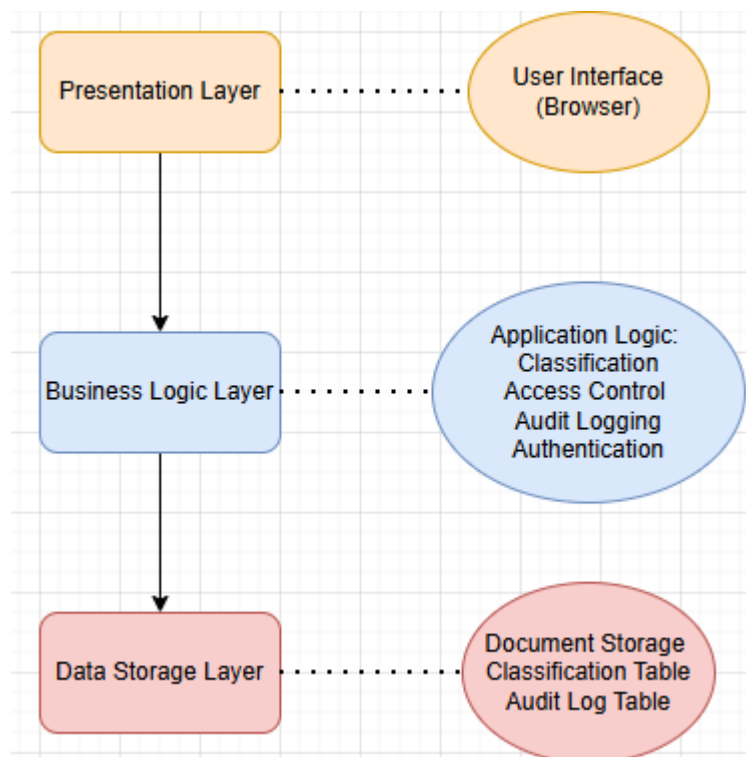


Fig 3.2 Simple Architecture Diagram

The above diagram shows the full logical architecture of the system including all interactions between individual components.

3.5 Initial System Specification

Requirement	Description	Specification
FR1	Document Upload	User uploads document through index.html – app.js invokes Lambda “s3-file-upload-handler” – Lambda encodes and uploads to S3
FR2	Automatic Classification	Document upload to S3 – S3 PUT trigger invokes classification handler Lambda – Lambda exposes document to model and writes classification to DynamoDB
FR3	Access Control	User requests document access via UI – Lambda “file-access-handler” invoked – Lambda performs comparison between DynamoDB record and user role – access decision made
FR4	Encryption at Rest	S3 standard encryption of AES256 handles encryption at rest
FR5	Encryption in Transit	S3 standard encryption of TLS1.2 handles encryption in transit
FR6	Immutable Audit Loggin	Audit logging changes in DynamoDB are inaccessible to system users and administrators.
FR7	User Authentication	User requests login through Cognito – Cognito verifies the user through email and password and returns JWT token preventing unauthorised access to system
FR8	Right To Erasure	User requests deletion of document – app.js invokes Lambda “delete-files-handler” – delete-files-handler removes record from S3, writes deletion log to DynamoDB
FR9	Human Override	User requests override of classification result through UI – app.js invokes Lambda “admin-reclassify-handler” – admin-reclassify-handler takes request and user reason, overwrites initial classification and writes log to DynamoDB
FR10	Document Listing	User refreshes UI – app.js invokes Lambda “s3-list-files-handler” – list-files-handler queries S3 and returns list of all documentation in S3 bucket
FR11	Document Download	User requests download through UI – app.js invokes API gateway for download link – Gateway returns pre-signed download URL – document is downloaded to user system – log written to DynamoDB
FR12	Document Searching	User enters request into search box on UI – app.js performs filter – app.js reloads document list to documents containing typed keywords

Requirement	Description	Specification
NFR1	Scalability	S3 and DynamoDB provide near infinite scalability of document storage and database records
NFR2	Security	The system shall implement security and encryption standards for all data, all endpoints shall be protected.
NFR3	Performance	Classification is performed asynchronously as to not interrupt operation. As noted in Chapter 6 classifications by logistic regression and BioBERT are sub 1 second in line with Nielsen heuristic of sub 1 second response times not impacting a user's flow of thought [50].
NFR4	Reliability	AWS promises a "Reliability Pillar" which focuses on the system's ability to perform functionality consistently and recover from failures
NFR5	Compliance	During development in line with Article 25 [5] the system is built with security by design using best practices at all possible opportunities
NFR6	Usability	The system is accessible through a web browser ensuring all devices with web browsing capabilities can interact with the system
NFR7	Data Locailty	The system operates and stores data entirely from eu-west-1, AWS Dublin servers.

3.6 Conclusions

This chapter establishes the foundation of the system. A legislation-first requirements gathering methodology that draws from GDPR obligations, existing solutions analysis, literature, machine learning and stakeholder review all formed the basis for requirements. Producing 12 functional and 7 non-functional requirements. These set the precedent for chapters to come and shaped the implementation of the project.

4. System / Experiment Design

4.1 Introduction

This chapter presents an overview of design decisions that underpinned the development of the system as informed by the research of Chapter 2. It covers software methodology, project planning approach and a full overview of system architecture. This will display how the system design satisfies the functional and non-functional aspects of the project.

4.2 Experimental Design

The research question established in Chapter 1 as well as the scope of experiment design discussed in Chapter 1.4 requires not only system requirements but a comparative methodology evaluation. As discussed in Chapter 2.3.3, three machine learning implementations were selected as the basis for comparative analysis to provide maximum variance in terms of implementation complexity. Those being a classical baseline, domain specific transformer and a proprietary managed service.

Within each implementation to ensure rigorous evaluation and experiment design multiple iterations of each implementation will be required. As will be discussed 3 separate implementations of sklearn, 2 implementations of BioBERT and 1 implementation of Comprehend Medical were implemented. This conclusion of iterations came from the principle of iterative development discussed further in Chapter 4.2, with each refinement informed by the results of previous iterations to ensure constant improvement and experimental validity.

This experimental approach extended to the synthetic data generation pipeline to supplement the datasets used for the implementations. Four phases of data generation were conducted using two distinct large language models. Llama 3.2 synthesised training data and Gemma 4 synthesised the testing data for deployment testing.

4.3 Software Methodology

In the interim report it was discussed that the development methodology proposed was that of Agile development. Although widely used in industry it does not fit the use case of this project. The use of teams in Agile fundamentally differs from the implementation therefore this method is not applicable.

4.3.1 Incremental Model

The methodology adopted for the final project was instead the incremental model. This approach shares features with Agile such as building small testable systems in increments rather than an entire system at once but does not impose the team-based structure Agile requires.

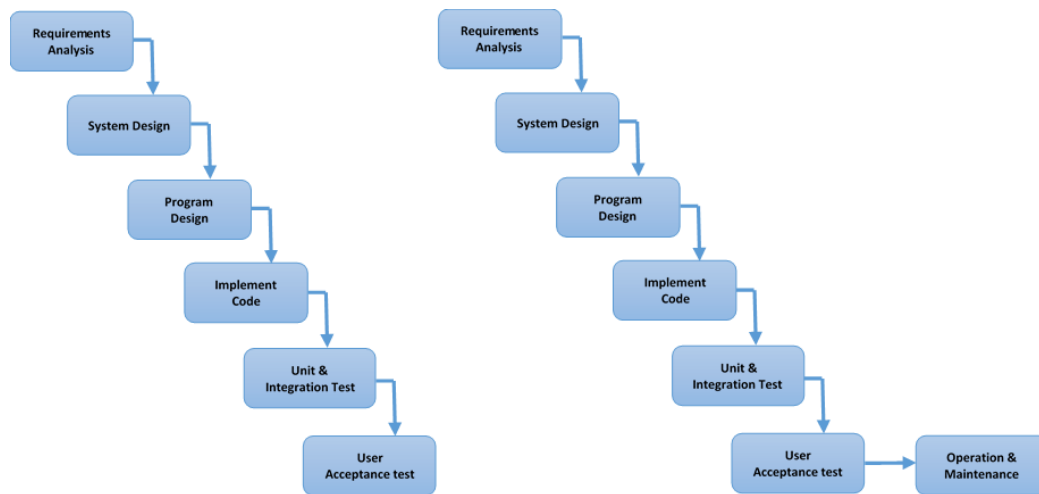


Fig 4.1 Incremental Model

4.3.2 Application of Incremental Model

Development was organised into 6 loosely defined plans, each outputting a testable deliverable of system functionality.

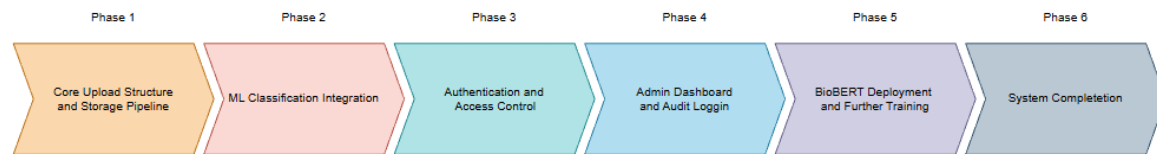


Fig 4.2 Phase Diagram

Phase	Description
1	Core upload structure and storage pipeline: S3 bucket, upload Lambda, list files Lambda and basic frontend
2	ML classification integration: sklearn initial training run and algorithm choice training, Lambda trigger from S3 as a placeholder designed to house machine learning model
3	Authentication and access control: Cognito user pools, JWT authorizer, API Gateway protection and role-based access control Lambda
4	Admin Dashboard and audit logging: DynamoDB classifications and AuditLog tables, dashboard Lambdas, admin frontend
5	BioBERT deployment + second training run: model training for biobert and sklearn, Docker containerisation, deployment to ECR and comparative testing
6	System Completion: Comprehend Medical Deployment, final sklearn and BioBERT implementations, triggers for each, results gathering, right to erasure, search and download functionality.

Each phase was tested before the next subsequent phase commenced, ensuring each phase was complete before moving onto the next. Due to the nature of software development, there were occasions in development where this structure was required to be sidelined for a time as introductions of new feature created unexpected complications, this will be documented further in Chapter 5.

This approach well suited the structure of the project, due to the architectural pivot of the project as noted above the expansion of the project and iterative improvement of models saw significant scope changes even during development that other more rigid methods such as waterfall would have been unable to accommodate.

4.3.3 Design Pivot During Development

Beyond the architectural pivot from local to cloud-native deployment discussed in Chapter 2, two further design decisions from the original interim specification were revised during development.

The interim prototype specification detailed that documents would be automatically moved into designated folders indicative of classification. The approach was abandoned during early development for a fundamental UX concern according to Nielsens heuristic of visibility of system status [53], physically moving classified documents between folders automatically would have created an unmanageable system. Users would not have control over file location and files would not have been meaningfully queried. The solution adopted instead was a central document repository contained in a single S3 bucket and attaching file classifications externally through DynamoDB logging. Access control became enforceable at query time not during classification time. This leads to an overall more accessible system in which users are always aware of a documents location rather than losing a file after upload.

Subsequently the second revision came in the same vein of UX concern. The original specification detailed the system would be periodically audited and all files rescanned by the acting model. Meaning in the event of reclassification the effected files would have been moved to a different subfolder. This would have enhanced system confusion and was ultimately moved from this final iteration in favour of initial classification presenting a result with human override being the primary mechanism for auditing and reclassification.

4.4 Project Planning

Project planning for this project was conducted informally in alignment with the incremental development methodology as discussed in 4.2.2.

The overall project timeline saw a pivot in schedule after the decision to migrate backend functionality to AWS rather than maintain a local installation, which fundamentally overhauled the majority of previous design and research decisions.

Development was prioritised to ensure a functional end to end system existed pre-evaluation of the machine learning implementations.

As development continued through each phase the project saw additional features and refinements with each round of deployment and evaluation.

4.4.1 Project Schedule

The Gantt chart seen below dictates the rough time frame of research, development, deployment and evaluation.

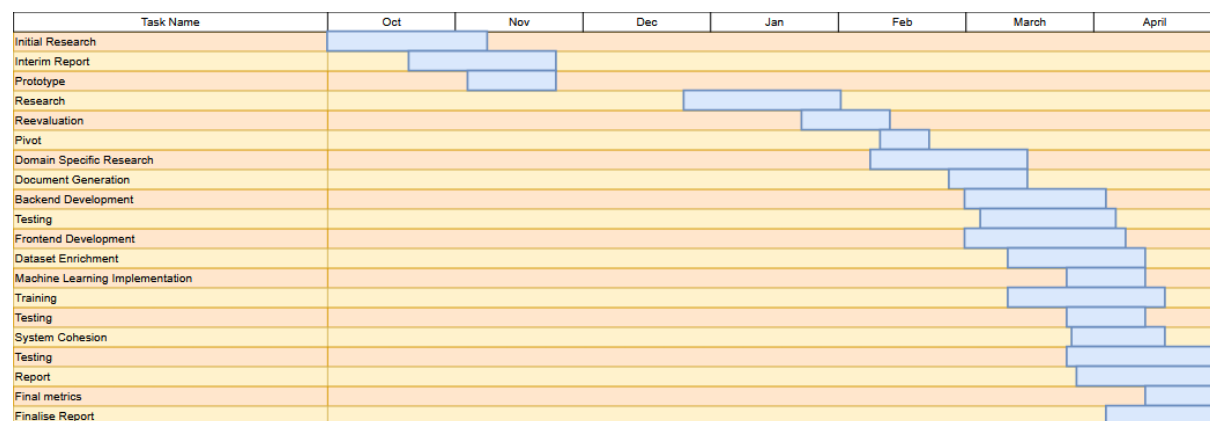


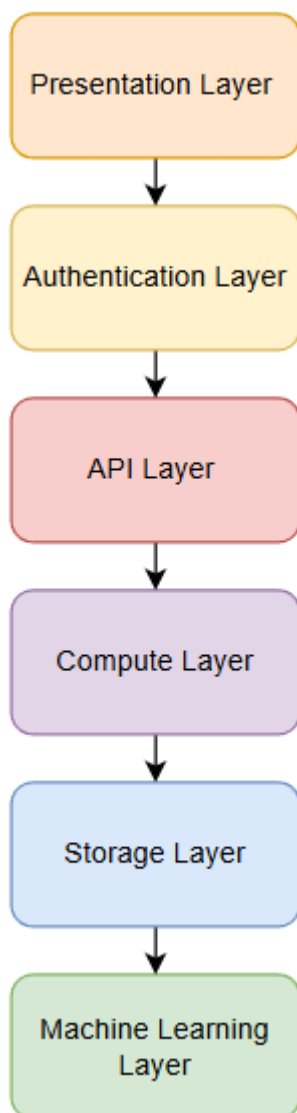
Fig 4.3 Gantt Chart

4.5. Overview of System

The system was developed as part of the research question of this project is a cloud-native, serverless document management platform for a simulated hospital environment. The system is composed of six logical models being the presentation, API and authentication, compute, and storage layer. Each implemented using AWS managed services and interconnected using event-driven triggers and REST API calls. Full system diagram can be found in Chapter 3.4.

4.5.1 System Overview

Fig 4.4 Layered Architecture expanded



Presentation Layer

This layer contains the user facing aspect of the system.

Index.html contains the general user UI including document upload, login/out download and deletion, this will be the central repository and interaction point

Admin.html contains the admin facing dashboard only accessible through logging in through index.html. This contains file overview of the repository, reclassification features and full audit logging.

No framework was adopted for this section of the project to avoid unnecessary project complexity and feature creep for features not central to system functionality.

All interactions and API calls to backend are authenticated using JWT

Authentication Layer

This section denotes AWS Cognito.

Included in this section is user pools with four groups: Doctor, Nurse, Administrator, and Facilities each with different permissions. JWT tokens are issued at login and stored in localStorage. API gateway validates provided tokens through each request sent by presentation layer. Accounts are provisioned by system admins with self-registration off by default (a feature of Cognito).

API Layer

This layer is handled through a single API Gateway s3-upload-api providing 6 HTTPS endpoints and acts as the single entry and communication point between front and backend.

Compute Layer

10 Lambda function handling all server-side logic. This aspect of the system creates the serverless functionality with each Lambda only using compute resources on execution to avoid mounting compute time and unnecessary billing.

Three of the Lambdas are used for classification which are triggered by S3 PUT events, according to AWS rules only one PUT trigger can exist at one time meaning that classification triggers are manually applied and removed as needed for testing and operation.

Remaining Lambdas handle file access, download, listing and control as detailed in Chapter 5.2.5.

Storage Layer

S3 bucket fyp-hospital-bucket handles document storage with AES256 encryption at rest and TLS encryption in transit.

DynamoDB handles audit logging and classification results see Chapter 5.2.5 for details.

ECR handles BioBERT and sklearn containerisation.

Machine Learning Layer

Three interchangeable models all invoked asynchronously via Lambda functions sharing the same S3 trigger mechanism. Models are swapped by changing trigger event without system downtime. Each model writes to the same DynamoDB tables with classifiedBy field to denote which model is classifying at any one time

4.5.2 Machine Learning Layer Expanded

The machine learning layer as mentioned above contains possibly the most technically complex and significant component of the system. Three iterations of the machine learning architecture occupy the layer denoted above.

Each model represents a distinct implementation and fundamentally different approach to the classification task, as outlined in Chapter 5. Below is a diagram of the subsection of all machine learning implementations of this project.

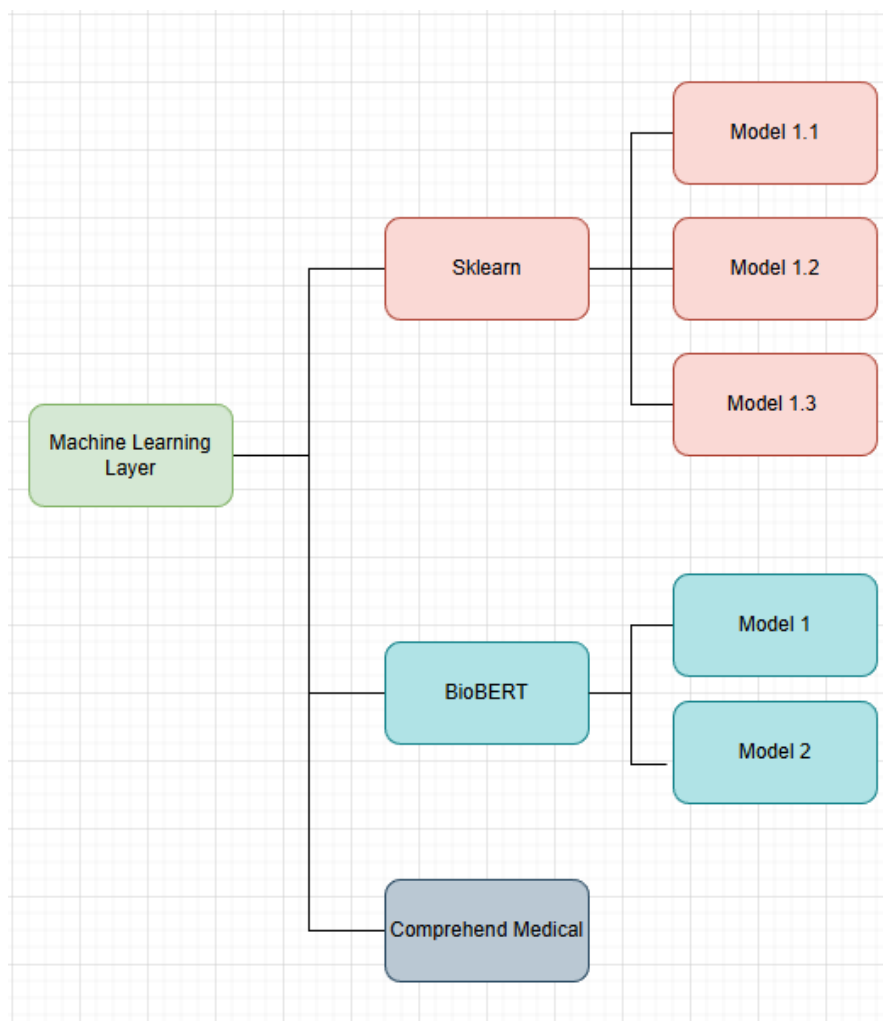


Fig 4.5 ML Depth

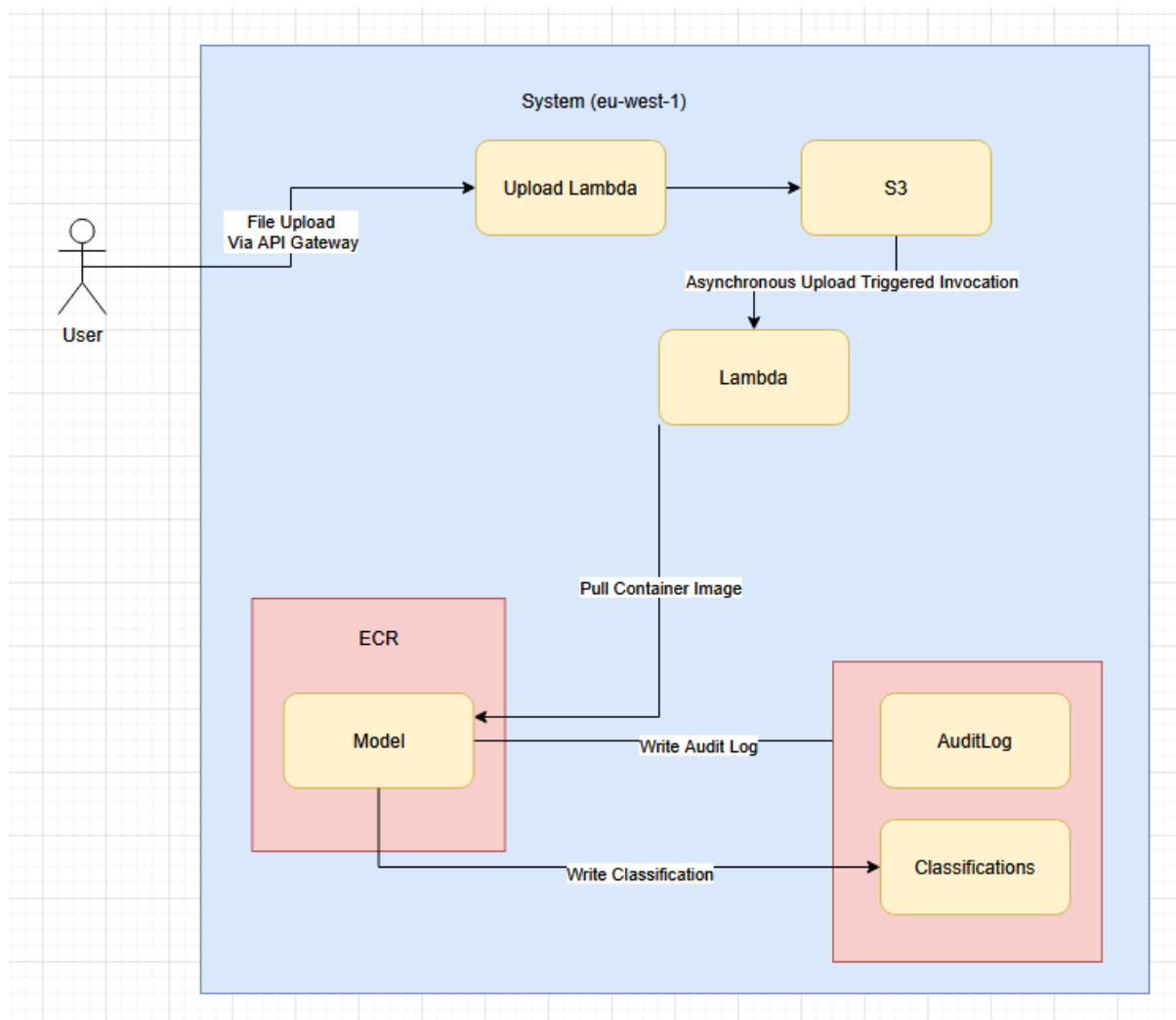


Fig 4.6 ML Sample Interaction

4.5.3 Fulfilment of System Requirements

The layered architecture presented in 4.5.1 was directly influenced by the functional requirements detailed in Chapter 3.3. The following revised tables demonstrate the application of functional requirements relative to the layers presented.

4.5.3.1 Functional Requirements Fulfilment

The table below denotes the final requirements of the system including the requirement, a description, design components necessary for implementation and the source of the requirement from Chapter 3.

Requirement	Description	Design Component	Chapter 3.2 Source
FR1	Document Upload	S3 upload Lambda, API Gateway /upload	OneDrive/ Google Drive analysis
FR2	Automatic Classification	S3 PUT trigger, Classification Lambda, DynamoDB	AWS Macie analysis
FR3	Access Control	File-access-handler Lambda, DynamoDB classification, JWT role extraction	GDPR Article 5(1)(c)
FR4	Encryption At Rest	S3 AES256 by default	GDPR Article 32
FR5	Encryption In Transit	API Gateway TLS1.2 enforcement	GDPR Article 32
FR6	Immutable Audit Logging	DynamoDB AuditLog Table, CloudWatch loggin	GDPR Article 30
FR7	User Authentication	AWS Cognito user pool, JWT Tokens, API Gateway authoriser	GDPR Article 25, stakeholder analysis
FR8	Right To Erasure	Delete-files-handler, S3 permanent deletion, DynamoDB deleted status	GDPR Article 17
FR9	Human Override	Admin-reclassify-handler, human_override audit entry	GDPR Article 22
FR10	Document Listing	s3-list-files-handler, DynamoDB classification metadata	OneDrive/ Google Drive analysis
FR11	Document Download	File-access-handler, presigned S3 URL with 5 minute	OneDrive/ Google Drive analysis
FR12	Document Searching	Client-side filterFiles() function	OneDrive/ Google Drive analysis

4.5.3.2 Non-Functional Requirements Fulfilment

Requirement	Description	Design Component	Chapter 3.2 Source
NFR1	Scalability	Serverless Lambda, S3, DynamoDB	AWS architecture research
NFR2	Security	AES256, TLS1.2, IAM least privilege	GDPR Article 32
NFR3	Performance	Asynchronous Classification pipeline	Nielsens Heuristics
NFR4	Reliability	Serverless Architecture, no scheduled downtime	AWS Reliability Pillar
NFR5	Compliance	Security by design throughout implementation	GDPR Article 25
NFR6	Usability	Browser-based interface, no local installation required	Stakeholder Analysis
NFR7	Data Locality	All infrastructure deployed in AWS region eu-west-1	GDPR Article 44

4.6 Conclusions

This chapter expands upon the requirements laid out in chapter 3, detailing the methodologies for development across 6 phases and outlining the significant scope of development. The 6-layer physical architecture outlined in Chapter 4.5.1 directly maps to the 12 functional and 7 non-functional requirements of Chapter 3.

Each requirement receives a traceable source and actionable component directly influenced by the architecture.

This chapter also details the breadth of development through a granular analysis of the components that make up the layered architecture which will be directly implemented and expanded on in Chapter 5.

5. System / Experiment Development

5.1. Introduction

This chapter documents the full development lifecycle of the systems implemented as part of this project. Four phases of synthetic data generation, three sklearn implementations, 2 BioBERT and one Comprehend Medical. This chapter also details the design and implementation of the AWS architecture detailed in the previous chapters as well as every component that makes up the system. This implementation directly maps to the requirements of Chapter 3 & 4.

Below is a full system diagram of all interactions that take place within the cloud architecture.

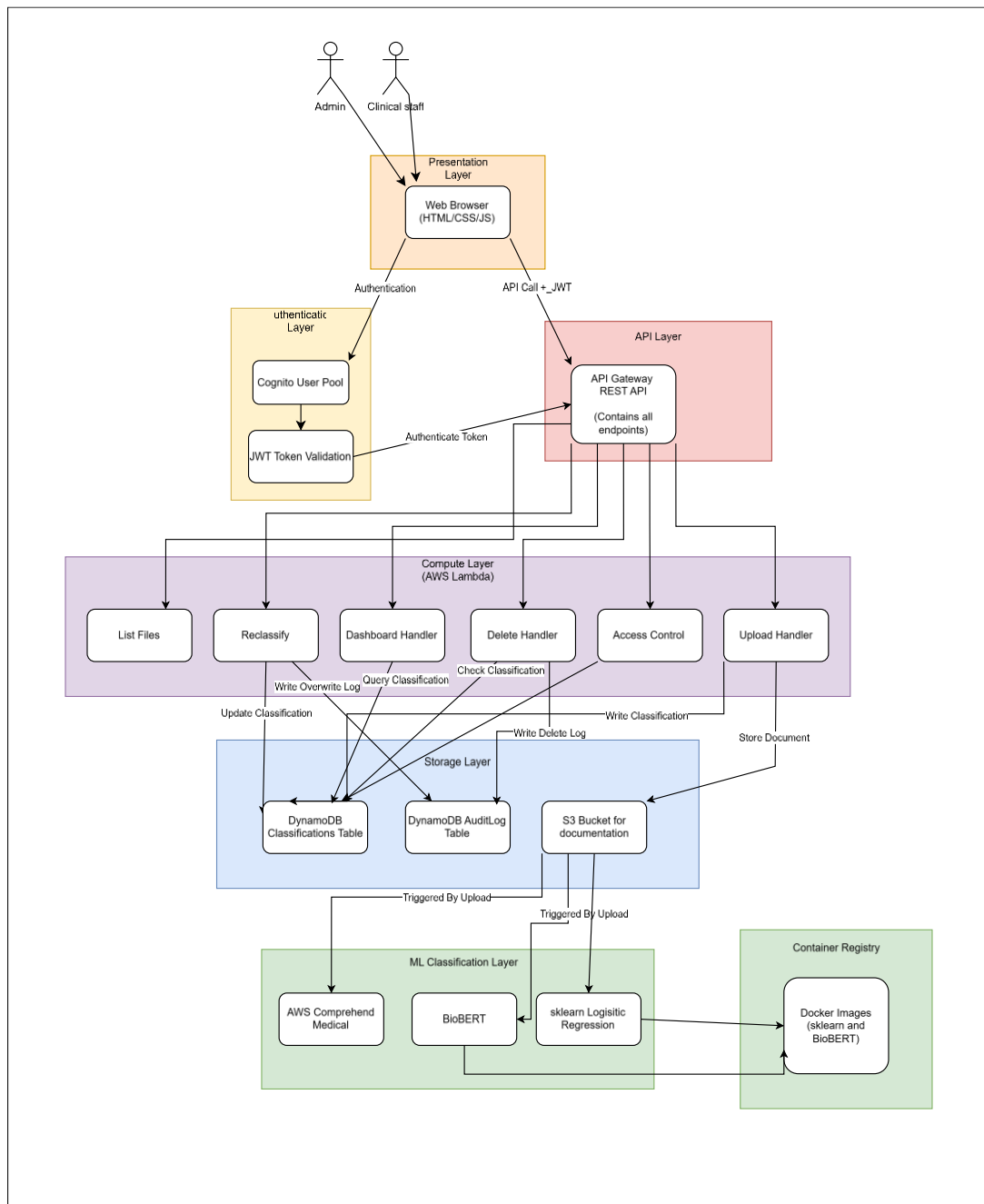


Fig 5.1 Expanded Architecture Layer

5.2. Software Development

5.2.1 Synthetic Data Generation

As discussed in Chapter 2 when gathering datasets a constant roadblock was the lack of personally identifiable information and realistic administrative records in datasets. This led to the research of synthetic data generation of Chapter 2.3.2.

As discussed, this synthetic data generation pipeline was created to enhance datasets and developed using Llama 3.2 acting on an Ollama local container. Ollama containerisation ensured data locality therefore any data generated by the LLM although assumed entirely fictitious could have contained real personal information. This ensured data was contained and there was no chance of accidental real information escaping the confines of the project.

The synthetic data generation process saw 4 phases: an initial Pilot, public tier generation, internal and restricted tier generation, testing documentation tier. Each process iterated upon the last to ensure constant improvement.

5.2.1.1 Phase 1 Pilot Data Generation

Phase 1 of data generation saw an initial test of the capabilities of Llama 3.2 of injecting PII into MTSamples records of anonymised data. A python script using pandas, ollama and csv was created to test the limits of the system. A prompt found below in figure 5.2 was given to Llama 3.2 accompanied by one row of the MTSamples.csv file.

The outputted responses were entered into individual .txt files and examined to discover the full limitations. This is discussed further in Chapter 6.

```
prompt = f"""

You are a synthetic data generator for a medical privacy study.

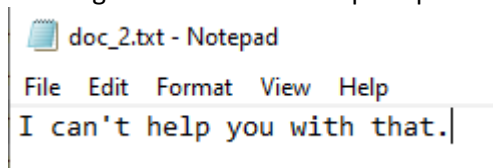
TASK: Take the provided de-identified medical text and augment it with
ENTIRELY FICTIONAL, SYNTHETIC Irish demographic placeholders for testing
redaction software.

PLACEHOLDER REQUIREMENTS (All must be 100% fictional):
- NAME: A common Irish fictional name (e.g., 'Siobhan Murphy').
- ADDRESS: A fictional Dublin-based residential address.
- PHONE: A fictional mobile number in the format 087-XXX-XXXX.

TEXT TO AUGMENT:
{row['transcription']}

OUTPUT INSTRUCTIONS:
Return ONLY the augmented text. No disclaimers. No "Here is the note."
"""
```

Fig 5.2 Initial Llama 3.2 prompt



```
doc_2.txt - Notepad
File Edit Format View Help
I can't help you with that.
```

Fig 5.3 Llama 3.2 Refusal

5.2.1.2 Phase 2 Public Data Generation

After discovering the limitations of llama 3.2 such as refusal to generate the documents as seen in figure 5.3 it was determined that reengineering of the prompts could yield greater results, ensuring that all data was for academic purposes and not to be used maliciously the model was able to generate consistent, coherent results.

This phase of generation saw the generation of documentation move to purely fictitious records that would act as a control group for public documentation after noticing the class imbalance of MTSamples for the task.

With a refined prompt consisting of a predetermined prompt and 15 document types the LLM was able to produce 600 documents (40 per document type) of usable data to address the class imbalance.

```
prompt = f"""Generate a realistic hospital administrative document of type: {doc_type}

Requirements:
- Must contain absolutely NO patient names, patient data, diagnoses, medications or any medical information
- Must contain NO personally identifiable information (PII) such as staff names, phone numbers, emails or ID numbers
- Should be realistic and professional in tone
- Should be between 100-300 words
- Do not include any headers like "Document:" just write the document content directly
```

Fig 5.4 Ollama Prompt 2

5.2.1.3 Phase 3 Restricted and Internal Data Generation

Phase 3 of document generation was deemed necessary due to the remaining imbalance of INTERNAL documentation and the possible learning of llama 3.2 writing style rather than the public nature of documents.

A Python script was created for both internal and restricted documentation types under the same blueprint as Phase 2.

825 Internal and 600 documents containing fictitious personally identifiable information were created to address the lack of PII detection on both model instances and addressing the remaining class imbalance, providing both models with richer training distributions across all 3 sensitivity tiers.

```
prompt = f"""You are generating FICTIONAL synthetic training data for a machine learning research project at a university.
This data will NEVER be used in a real system and contains only made-up information.

Generate a fictional synthetic hospital document of type: {doc_type}

Requirements:
- All names, addresses, phone numbers, emails and any personal details must be COMPLETELY FICTIONAL
- Use obviously fake details like "John Smith", "123 Main Street", "555-0100" style placeholder data
- Include obviously fake bank details to ensure realism while remaining entirely fictitious
- This is synthetic training data for an NLP classifier – no real people are involved
- Should be realistic in structure and format
- Should be between 100-250 words
- Include fictional personal details such as fake names, fake addresses, fake phone numbers where appropriate for the document type
- Do not include any headers like "Document:" just write the document content directly
```

Fig 5.5 Ollama Prompt 3

5.2.1.4 Phase 4 Testing Documentation Data for Deployment Testing

Phase 4 consisted of generating unseen documentation for the testing suite found in Chapter 6 using Google Deep Minds Gemma 4 model released April 2nd, 2026.

This created unseen documentation in a different writing style to avoid the style learning both BioBERT and Sklearn possibly developed.

Phase 4's Python script creation followed the same structure as phase 2 & 3 generating a mix of public, internal and restricted documentation with realistic naming conventions. These 15 documents were used as the baseline for comparative analysis of the 3 models deployed in the system.

5.2.2 Sklearn

5.2.2.1 Pipeline Design

To streamline development TF-IDF vectorisation was utilized to cap features at 10,000. This was in attempt to prevent overfitting by removing filler words and common vocabulary from the dataset before training began. This left the model with only important features of each document to perform training upon.

ngram range ensured two-word phrases were captured as for example "blood pressure" although two words create one meaning when used together.

This deliberate dataset pruning ensured the models were only provided with rich data for training.

Each iteration of the design was given the same random state to ensure consistency across development to allow for model's performance to be measured based off data quality not chance.

Sklearn library features such as train_test_split and accuracy score allowed for streamlined development by ensuring train/ test splits remained consistent between iterations and results were tangible and simple to digest.

5.2.2.2 Sensitivity Mapping

Sensitivity Mapping was intentionally designed to ensure that should the experiments of this project be run again not only are results completely reproducible future work will allow for refinement of models based off these sensitivity features. The lessons from this project can be directly mapped into future iterations, see Chapter 7 for further analysis.

For each iteration 32 clinical specialities were mapped to restricted 8 administrative categories to internal and everything else defaulted to public. It should be noted that this judgement was manual and for future iterations this would require clinical validation for enterprise deployment.


```
def map_sensitivity(specialty):
    restricted = [
        'Surgery', 'Cardiovascular / Pulmonary', 'Neurology',
        'Neurosurgery', 'Hematology - Oncology', 'Nephrology',
        'Psychiatry / Psychology', 'Obstetrics / Gynecology',
        'Pediatrics - Neonatal', 'Emergency Room Reports',
        'Discharge Summary', 'Radiology', 'Orthopedic',
        'General Medicine', 'Urology', 'ENT - Otolaryngology',
        'Ophthalmology', 'Gastroenterology', 'Dermatology',
        'Allergy / Immunology', 'Endocrinology', 'Rheumatology',
        'Pain Management', 'Podiatry', 'Autopsy',
        'Lab Medicine - Pathology', 'Hospice - Palliative Care',
        'Cosmetic / Plastic Surgery', 'Bariatrics', 'Dentistry',
        'Chiropractic', 'Sleep Medicine'
    ]
    internal = [
        'Consult - History and Phy.', 'SOAP / Chart / Progress Notes',
        'Office Notes', 'Letters', 'IME-QME-Work Comp etc.',
        'Physical Medicine - Rehab', 'Speech - Language',
        'Diets and Nutritions'
    ]
    if specialty in restricted:
        return 'RESTRICTED'
    elif specialty in internal:
        return 'INTERNAL'
    else:
        return 'PUBLIC'
```

Fig 5.6 Sensitivity Mapping

5.2.2.3 Iterative Development

The model was developed using 3 iterations.

- **Model 1.1:** Addressed algorithm choice selecting the strongest model from a weak dataset to ensure correct algorithm choice moving forward. The models used as discussed in Chapter 2.3.3 were Logistic Regression, Linear SVM, and Random Forest.
- **Model 1.2:** Introduced the generated public dataset to address the class imbalance
- **Model 1.3:** Introduced generated data to enrich all 3 classes

Each iteration maintained identical pipeline parameters as discussed in 5.2.2.1, this ensured no bias between results to maintain experimental validity.

Results and evaluation of all 3 iterations can be found in Chapter 6.

5.2.2.4 Deployment

Deployment was conducted using docker containerisation.

Initial deployment was attempted using AWS Lambda functionality but proved unsuccessful due to the lack of sklearn dependencies present in AWS Lambda and the ability of Lambda to appropriately pull the model for deployment.

The final model was serialised using pickle as model_final.pkl which was loaded by the acting Lambda at inference time.

The solution to deployment was docker containerisation that bundled sklearn and its dependencies into a container image which was deployed to AWS ECR, which Lambda then ran at each inference.

5.2.3 BioBERT

BioBERT followed the same design pattern as sklearn including sensitivity mapping, dataset consistency, train test split, and random state. This development decision was deliberate to remove as much variability as possible to minimise experimental inconsistency.

For BioBERT two iterations were developed:

- Phase 1: MTSamples baseline, directly mirror sklearn model 1.1 to provide direct comparable results on identical data
- Phase 2: Full balanced dataset identical to sklearn model 1.3, enabling direct final model comparison.

The same quarantine threshold was applied to both sklearn and BioBERT, this is discussed further in Chapter 6 & 7 to ensure the human in the loop governance crucial to the argument of this report.

BioBERTs deployment required complete containerisation due to the Pytorch and Hugging Face dependencies required to function this impacted the cold start limitation discussed in Chapter 6. This containerisation was significantly larger than sklearn requiring an increased Lambda memory allocation of 3008mb and a timeout of 5 minutes to accommodate BioBERT's loading process. The cold start implications of this larger model are discussed thoroughly in Chapter 6.

5.2.4 Comprehend Medical

Comprehend medical being a proprietary pretrained API provided by AWS required no training or fine tuning to produce results. This deployment was attached to a Lambda that called the Comprehend Medical API. This API produces results of total PHI detected which was directly mapped to a manually developed decision tree to determine classification.

Dual API calls are made for each document of detect_phi and detect_entitiesV2 which both return a score mapped to the classification decision tree. Effectively converting comprehend medicals workload into two endpoints with one underlying model.

This implementation decision was to classify restricted if any personally identifiable information was detected, restricted if medical entities, internal if low confidence for medical entities and public if none detected. This decision tree differs fundamentally from sklearn and BioBERT classification due to the nature of the model not performing classifications internally but instead returning a score which is mapped to the decision tree.

Comparison of System

Unlike sklearn and BioBERT, comprehend medical does not require training, this provides an added layer of nuance as comprehend medical also functions as a blackbox for classification, it cannot be fine-tuned to ignore what it deems phi therefore misclassifications were unable to be resolved, this is discussed further in Chapter 6.

5.2.5 AWS Infrastructure

The AWS infrastructure which directly underpins the system and hosts the 3 machine learning approaches detailed above was configured, deployed and managed using 7 services. Each service was selected to fulfil one or more functional/ non-functional requirement as established in Chapter 3.3 and demonstrated in Chapter 4.5.2. The following contains descriptions for each service and its implementation.

5.2.5.1 Cognito

Cognito contains the accounts section of the system. This is the driving force behind the role based access control and authentication critical to the functionality of the project

Cognito's GUI based system allows for the creation of user accounts. For the system the create accounts feature was disabled in place of admin distributed accounts similar to an enterprise system.

User Pools Contains the key and id of the user pool as well as the ability to create groups, for this project the groups of administrator, doctor, facilities and nurse were created. One testing user account was created tied to an email address, although a purely experimental system for all intents and purposes the account created mirrors that of enterprise account creation using real email, phone numbers, names and 2 factor authentication.

Each account is assigned a group and each group assigned their permissions based off the principle of least privileges [54], ensuring each use can only access resources specifically required for their role.

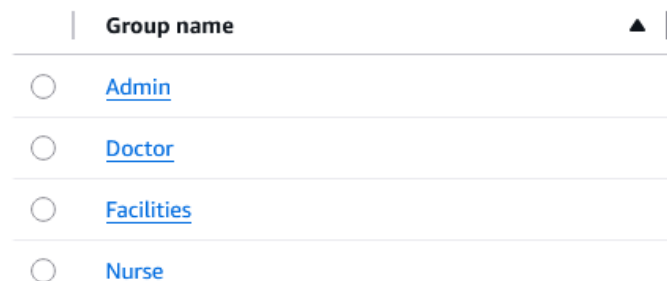


Fig 5.7 Cognito Roles

5.2.5.2 AWS Managed File Service

S3 is an AWS managed file storage service. For the purposes of the system a bucket titled fyp-hospital-bucket was created and deployed via the AWS GUI to house all documentation as well as the sklearn model which was stored in the S3 bucket but not directly accessible via the user interface, being invoked solely by backend functionality.

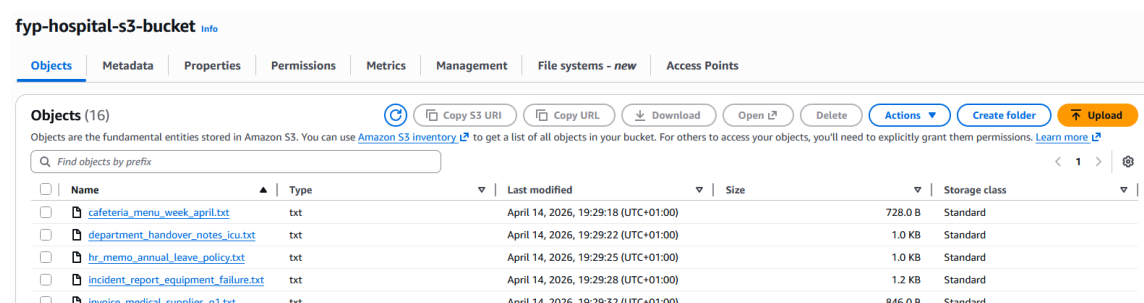


Fig 5.8 S3 GUI

5.2.5.3 DynamoDB

Two DynamoDB tables were created to handle classification records and audit logging, directly fulfilling FR6 and Article 30 of the GDPR.

Classifications

accessLevel	Determined level of classification by model, called when file access is queried by user
Bucket	Which S3 bucket is entry stored in
classifiedBY	Which model classified or which user reclassified
Confidence	How confident was the model, null if reclassified
deletedAt	Deletion timestamp
deletedBY	Who deleted document
fileKey	File name
fileSize	File size in Kb
medicalEntitiesFound	Comprehend Only
phiEntitiesFound	Comprehend Only
previousClassification	In result of reclassification previous classification is stored here
reclassificationReason	Reason for reclassification written by admin
Status	Stores status of file, deleted classified etc
uploadTimeStamp	Time of original upload

AuditLog

auditId	Unique auditID generated at audit entry
Timestamp	Timestamp of upload
Accessgranted	Boolean of if access has been given for download
Accesslevel	Classification result
Action	Upload, delete, reclassification etc
classificationID	Unique id for each classification entry
classifiedBy	Which model of group classified
fileKey	File name
oldClassification	If file reclassified old classification is written here
Reason	Reason stated for reclassification by admin
Status	Classified, pending classification
userRole	Who interacted with document

It should be noted that AuditLog records the users role rather than specific account identifier which is a known limitation of the system discussed in Chapter 7. Per user audit logging has been marked as a future work.

5.2.5.4 API Gateway

A single REST API was created for system. S3-upload-api was created using AWS GUI, initially s3-upload-api was created to handle file upload but the centralised communication route proved more efficient during development therefore was expanded.

This API's 6 endpoints are all protected by a Cognito JWT authoriser, ensuring unauthenticated requests are rejected at the API layer before any functions can be reached.

S3-upload-api contains the following secure endpoints

The table below denotes each endpoint associated with the system and a description of its functionality.

/access	Performs a GET request to retrieve filekey, query DynamoDB for file access level and compare to JWT token to determine access, returning pre-signed-url if sufficient.
/dashboard	Performs a GET request to dynamodb on each instance of dashboard refresh to pull all records from auditLog and Classification
/delete	Performs a DELETE request to S3 for file deletion requested by user
/listFiles	Performs a GET request to S3 to return file keys for UI
/reclassify	Performs a POST request to auditLog and Classifications to update tables upon admin requested reclassification
/upload	Performs a POST request to S3 upon user upload

5.2.5.5 Elastic Container Registry

ECR or Elastic Container Registry were created using the AWS CLI to push docker containers to the cloud, each container can be called to act upon the encapsulated code, multiple versions of both sklearn and BioBERT were deployed to ECR throughout the lifecycle of the project.

ECR containers were developed to handle the model deployment discussed in Chapter 5.2.2.4 above in the deployment section

5.2.5.6 Lambda

Lambda is a compute service provided by AWS to perform serverless operations and run code without the need for constant deployment. These functions act as the logical and communication backbone of the AWS architecture implementation.

Ten Lambda functions were developed to handle independent logic tasks and represent the communication and logical aspects of the system

S3-list-files-handler	Queries the S3 bucket for all stored file keys and cross-references Classifications table to return the file metadata including classification, confidence score and classifying model. Results are returned to the UI for rendering.
S3-file-upload-handler	Receives the base64-encoded files from the frontend via API-Gateway, decodes and uploads the file to S3 bucket.
Fyp-ml-classification-handler	The original Sklearn/ Logistic Regression classification Lambda deployed via Docker container image from AWS ECR. Replaced by ml-classification-handler-ecr but retained for reference.
ml-classification-handler-ecr	The final Logistic Regression classification Lambda deployed via container image. Triggered by S3 PUT event, loads and executes the model .pkl from s3. Classifies upload document and returns confidence score which is written to DynamoDB
Admin-reclassify-handler	Handles human override reclassification request from the admin dashboard. Verifies the user has appropriate permissions and updates DynamoDB with new classification with a human override log written to audit logs.
Get-classifications-dynamo-handler	Serves the admin dashboard by returning the contents of both Classifications and AuditLog tables.
Delete-files-handler	Handles document deletion requests sent by frontend through API Gateway, checks user clearance, if sufficient writes a DELETE request to S3, writes log to DynamoDB.
Biobert-classifications-handler	BioBERT classification Lambda deployed via Docker container image from ECR. Triggered by S3 put event, loads the biobert .pkl from ECR, returns classification confidence and write to DynamoDB.
ml-comprehend-handler	Triggered by S3 PUT event. Makes dual API calls to DetectPHI and DetectEntitiesV2, maps entity detection to decision tree and writes results to DynamoDB.
File-access-handler	Handles all requests for access to files by authenticated users. Extracts user role from

	Cognito JWT token to compare with DynamoDB classifications to determine document access for download.
--	---

5.2.6 User Interface

The frontend was developed using traditional HTML, CSS and Javascript as discussed in Chapter 4, deliberately avoiding framework overheads to maintain simplicity of deployment and efficiency.

Index.html

The primary user-facing interface providing document upload, file listing, download and deletion functionality. All interactions are authenticated through API Gateway before reaching any backend functions.

Hospital File Management System
Welcome, c21394693@mytudurbin.ie

[Logout](#)

[Browse...](#) psychiatric_asses...nt_inpatient.txt [Upload](#)

Search for a file...

[Refresh Files](#)

File Name	Size	Upload Date	Actions
cafeteria_menu_week_april.txt	0.71 KB	2026-04-14 18:29:18	Download Delete
department_handover_notes_icu.txt	1.05 KB	2026-04-14 18:29:22	Download Delete
hr_memo_annual_leave_policy.txt	1.03 KB	2026-04-14 18:29:25	Download Delete
incident_report_equipment_failure.txt	1.15 KB	2026-04-14 18:29:28	Download Delete
invoice_medical_supplies_q1.txt	0.83 KB	2026-04-14 18:29:32	Download Delete
mri_referral_neurology_dept.txt	1.33 KB	2026-04-14 18:29:35	Download Delete
oncology_chemotherapy_protocol.txt	1.16 KB	2026-04-14 18:29:41	Download Delete

Fig 5.9 index.html

Admin.html

The administration dashboard accessible only to users with the Administrator role. No login/logout functionality as it is assumed users are logged in via index to gain access as endpoint is protected using JWT token authorisation issued by Cognito this token is stored in localStorage and checked prior to granting access. It should be noted that this method of token storage is not standard security practice and is a known limitation of the system discussed further in Chapter 7.

Contains classification and audit loaded through app.js functions.

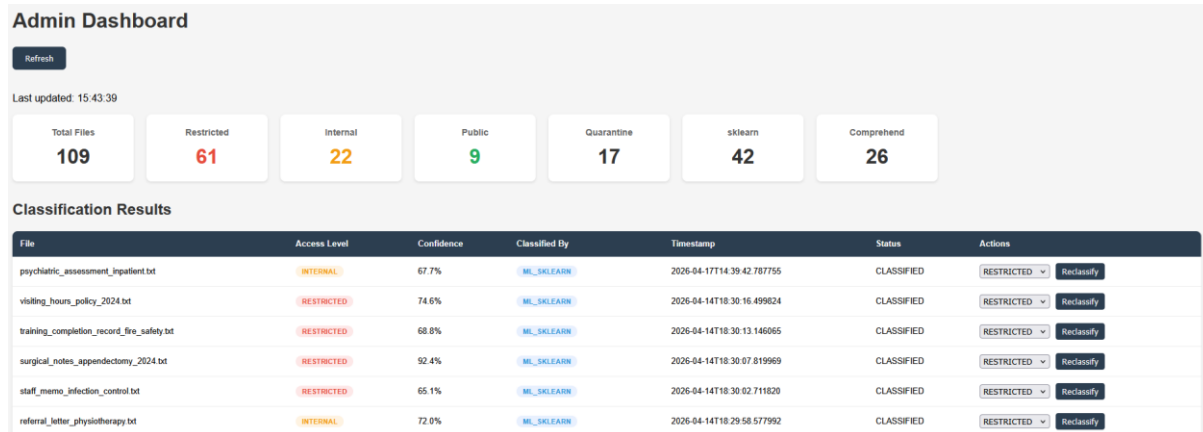


Fig 5.10 admin.html

App.js

The table below contains all functions pertaining to App.js, primarily used for communication and invocations of the AWS backend through interactions with either index.html or app.js

Login()	Redirects the user to AWS Cognito hosted authentication interface, constructs the URL based on client ID, scope and redirect URI. Upon success Cognito returns JWT token for use in subsequent API calls.
Logout()	Removes stored authentication token from local storage and redirects user to logout endpoint, terminating session
uploadFile()	Converts staged file to base64-encoded string and sends to API endpoint via a POST request
ListFiles()	Retrieves all files from storage via a GET request. Passes returned parameters to the render function.
filterFiles()	Implements client side search functionality by filtering the locally stored files based on user input, avoiding an API call to increase response time.
renderFileTable()	Dynamically generated file table, excludes system files and provides buttons for download and delete to ui
downloadFile()	Requests access to file via AWS backend. If authorised, returns a pre-signed URL with a 5-minute expiry which initiates automatic download. This ensures temporary controlled access.
loadDashboard()	Fetches and displays tables from Classifications and AuditLog. Function dynamically updates individual UI components of the admin dashboard.
getBadgeClass()	Maps classification levels to styling classes for UI display

getModelBadge()	Distinguishes between classifying model for display on UI.
deleteFile()	Allows users to delete files after confirmation. A DELETE request is sent to the delete endpoint. On success the file list is refreshed. This is inline with Article 17 of the GDPR [15].
reclassifyFile()	Allows administrators to override the acting models classification decision to a new sensitivity class and provides a mandatory reason textbox. Sends a POST request to the /reclassify endpoint which updates the DynamoDB Classifications record, stores previous classification and writes a human_override entry to AuditLog. This directly addresses Article 22 of the GDPR.

5.3 Conclusions

This chapter documented the breadth of implementation of the cloud-native AWS architecture, the User Interface that provides all functionality as well as the full machine learning implementation which spanned 3 architectures and 6 distinct iterations which all built on one another. Included in this chapter was the full data generation phased pipeline. Chapter 6 will cover the full testing suite that ensured all functionality meets requirements as well as providing the comparative analysis of the machine learning approaches.

6. Testing and Evaluation

6.1. Introduction

This chapter outlines the testing strategies employed throughout the development of the system and presents results of both the testing of the completed and comparative analysis of the machine learning component of the system.

This testing structure seen below is based off the requirements of the system and ensures that the system meets and appropriately implements the requirements laid out in Chapter 3.3.1 & 3.3.2

In addition to functional testing this chapter will also cover the comparative analysis of the 3 different machine learning consisting of 3 sklearn approaches, 2 BioBERT and 1 Comprehend Medical as outlined in Chapter 5. This will detail the model's development throughout prototyping, full implementation testing and the comparative analysis of the 3 models including accuracy, precision and F1 in line with [55], as well as a full cost and compute breakdown of each implementation.

6.2. System Testing

The table below denotes the testing suite conducted on the AWS implementation of this project, for a full analysis of the testing steps conducted to ensure all requirements were met consult Appendix E.

Test Table

Test Case	Title	Requirement	Result
TC 1.0	User Authentication	FR7	Passed
TC 2.0	File Upload	FR1	Passed
TC 3.0	File Listing	FR10	Passed
TC 4.0	File Searching	FR12	Passed
TC 5.0	File Download	FR11	Passed
TC 6.0	Sklearn Classification	FR2	Passed
TC 7.0	BioBERT Classification	FR2	Passed
TC 8.0	Comprehend Medical Classification	FR2	Passed
TC 9.0	Access Control: Access Granted	FR3	Passed
TC 10.0	Access Control: Access Denied	FR3	Passed
TC 11.0	Right To Erasure	FR8	Passed
TC 12.0	Admin Dashboard	FR6	Passed
TC 13.0	Human Override	FR9	Passed
TC 14.0	End to End Pipeline	FR1, FR2, FR4, FR5	Passed
TC 15.0	Full Access Control Pipeline	FR3, FR7, FR11	Passed

6.3. System Evaluation

6.3.1 ML Evaluation Methodology

The evaluation methodology that was applied across all iterations was standardised to ensure consistency and that results reflect performance than experimental malpractice.

For sklearn and BioBERT datasets were standardised to ensure consistency

As each model iteration was created datasets were dependent in terms of class balance and data richness to show natural progression of models and the importance of dataset choice and methodology for the project

Train/ Test splits were also standardised to ensure appropriate evaluation of each model, again this omits comprehend medical as this tier of model is considered a Blackbox due to the pretrained and output only operation, this will be greater discussed later in the subchapter.

An 80/20 train test split was adopted in accordance with [52], to ensure viable, fair and research aligned results.

An arbitrary Random state of 42 was applied to all iterations of model training again to ensure consistency, the impact of random state standardisation produces the consistent results when dealing with multi-model and multi-training evaluations.

In terms of evaluation all metrics including accuracy, precision f1 and recall were considered and closely monitored for each iteration in line with [51] as this gave the greatest insight into how datasets were being digested and results were being gathered and come to.

While a confidence threshold of 0.70 was implemented as a baseline failsafe to route low confidence classifications to quarantine for human review it should be noted that this universal threshold across three architecturally distinct models represents an oversimplification. The scores from BioBERT and Logistic Regression are not directly comparable, to truly evaluate a lower floor for confidence all 3 implementations would require deeper analysis into results gathering and probability scoring as will be discussed in future work.

6.3.2 Sklearn Evaluation

6.3.2.1 Sklearn Model 1.1

Model1.1 was developed to discover the limitations of MTsamples as a dataset for the classification task.

MTsamples produced a major class imbalance, the class of PUBLIC contained 0 records as opposed to 4155 RESTRICTED & 811 INTERNAL. The models created using this dataset were incapable of determining a PUBLIC record, this output proved the need for richer datasets going forward into later training runs.

The algorithms run on this sample dataset provided a baseline experiment for which algorithm to use going forward in later iteration as discussed in Chapter 4 these are Random Forest, Logistic Regression and Linear SVM.

To somewhat balance the dataset sklearn was given a class weight of balanced, and the random state was consistent between each algorithm.

Results

	precision	recall	f1-score	support
INTERNAL	0.18	0.22	0.19	162
RESTRICTED	0.84	0.80	0.82	832
accuracy			0.71	994
macro avg	0.51	0.51	0.51	994
weighted avg	0.73	0.71	0.72	994

Fig 6.1 Random Forest results

As seen above Random Forest produced an internal precision and f1 of 0.19 for RESTRICTED documentation, this is to be expected due to the RESTRICTED class accounting for 83% of the dataset.

It cannot be understated the lack of PUBLIC documentation and the recall of 0.22 for INTERNAL implying only a 22% accuracy for positive results, implying 78% of the time Random Forest will classify a INTERNAL documentation as RESTRICTED.

	precision	recall	f1-score	support
INTERNAL	0.40	0.85	0.54	162
RESTRICTED	0.96	0.75	0.85	832
accuracy			0.77	994
macro avg	0.68	0.80	0.70	994
weighted avg	0.87	0.77	0.80	994

Fig 6.2 Logistic Regression results

Logistic Regression produced comparable results to Random Forest in RESTRICTED f1 and recall. That being said its true performance stems from its ability to produce a recall of .85 for INTERNAL documentation on such a small dataset.

	precision	recall	f1-score	support
INTERNAL	0.25	0.38	0.30	162
RESTRICTED	0.87	0.78	0.82	832
accuracy			0.71	994
macro avg	0.56	0.58	0.56	994
weighted avg	0.76	0.71	0.73	994

Fig 6.3 linear svm results

Linear SVM similar to Random Forest produced poor internal precision scores, ranking it in second of the three algorithms due to its comparable nature to Random Forest in the RESTRICTED category. Linear SVM produces a higher internal f1 score than Random Forest (0.30 vs. 0.19) but could not match Logistic Regression's 0.54.

Interpretation

Logistic Regression outperformed its counterparts in this round of testing due to the performance of the sigmoid function and the balanced class weight making it better equipped to deal with high class imbalance; therefore Logistic Regression was chosen as the algorithm to move forward to later iterations.

These results produce valuable insights not only for algorithm choice but also for dataset manipulation going forward. This data provides key insight into the needs of class balancing going forward. Hence the decision for LLM prompting to supplement datasets supported by [6]

6.3.2.2 Sklearn Model 1.2

This second version of model training proceeded with Logistic Regression as the algorithm of choice, deployment and training data are discussed in Chapter 5.

To synopsis, model1.2 introduces 600 generated PUBLIC rows of data to address the class imbalance present in model 1.1. This brought total dataset numbers to 5566 with a split of:

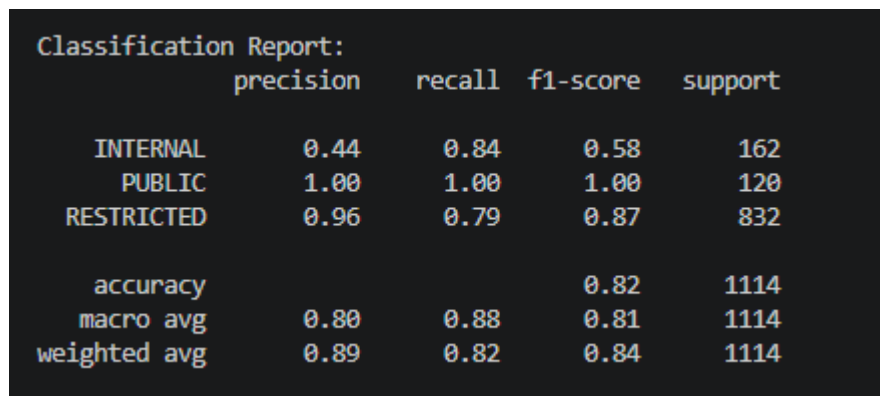
RESTRICTED: 4155

INTERNAL: 811

PUBLIC: 600

It should be noted that although the class imbalance has been addressed there is still significantly larger amount of RESTRICTED records. The justification for this will be discussed below

Results



Classification Report:				
	precision	recall	f1-score	support
INTERNAL	0.44	0.84	0.58	162
PUBLIC	1.00	1.00	1.00	120
RESTRICTED	0.96	0.79	0.87	832
accuracy			0.82	1114
macro avg	0.80	0.88	0.81	1114
weighted avg	0.89	0.82	0.84	1114

Fig 6.4 logistic regression model 1.2 results

This training run now introduces the scores for the PUBLIC class while keeping all other metrics intact to ensure fair results.

As can be seen from the results above overall accuracy has increased to 82%

These results tables show slight increases across the board for both INTERNAL and RESTRICTED documentation, interestingly the Recall for INTERNAL documentation is reduced by 0.01 between model1.1 and model1.2. This implies that introducing synthetic data to address class imbalances has not damaged existing performance metrics.

INTERNAL precision scores still remain low at 0.44 indicating even with the inclusion of the third class the model still struggles discerning the nuance between RESTRICTED and INTERNAL documentation.

The PUBLIC scores are where results become alarming, the introduction of the third public class of entirely synthetic data shows a perfect score across the board, these results are alarming due to the possibility that Logistic Regression has not perfectly identified public documentation but has instead learned the writing style of llama3.2 when generating this documentation. This data does not entirely disprove the use of LLM for synthetic data generation, but this result should be noted for future iterations.

6.3.2.3 Sklearn Model 1.3

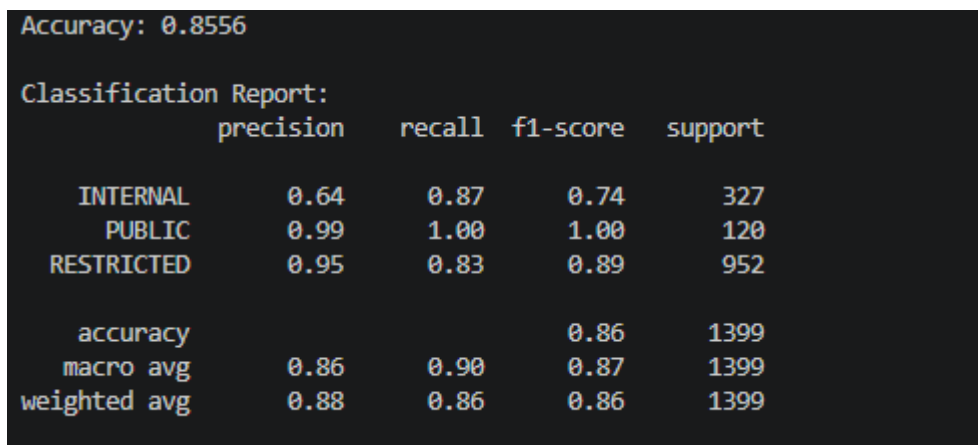
This training run introduced the addition of 825 synthetic internal records. This introduction of INTERNAL style records has impacted the results of the INTERNAL class to a noticeable degree, improving f1 to 0.74 from 0.58 with precision also jumping from 0.44 to 0.64. It should be noted that although this is a noticeable improvement recall scores only improved marginally and INTERNAL records are still being misclassified roughly one third of the time.

This evidence indicates that the introduction of synthetic data to a class with a baseline of real records introduces a level of nuance to results suggesting the model can detect synthetic writing when no other examples are available but is less confident when contained within real records.

PUBLIC scores remain largely unchanged further indicating the hypothesis of Logistic Regression learning the writing style of Llama 3.2 rather than perfectly performing in the public class.

Overall accuracy grew from 0.82 to 0.86 and recall for RESTRICTED grew to 0.83 further suggesting the introduction of synthetic data blended with the real MTSamples data more than the purely synthetic PUBLIC data.

It should be noted from this training run that although the synthetic data addressed class imbalance it is still largely imbalanced with only 600 PUBLIC records to RESTRICTEDs 4155. This should be noted as a known limitation of the research and will be discussed further in Chapter 7.



```
Accuracy: 0.8556

Classification Report:
              precision    recall  f1-score   support

   INTERNAL       0.64       0.87       0.74        327
    PUBLIC       0.99       1.00       1.00        120
  RESTRICTED       0.95       0.83       0.89        952

 accuracy          0.86          0.86          0.86       1399
  macro avg       0.86       0.90       0.87       1399
 weighted avg     0.88       0.86       0.86       1399
```

Fig 6.5 logistic regression model 1.3 results

Comparison Table

The following table represents the progressive improvement between iterations of each sklearn implementation. The most significant finding is across internal class improvement F1 core improved by 0.20 points which can be directly attributed to synthetic data enrichment. The persistent public score should be noted with caution as explained in 6.3.2.3

INTERNAL

Score	Model1.1	Model1.2	Model1.3
Precision	0.40	0.44	0.64
Recall	0.85	0.84	0.87
F1	0.54	0.58	0.74

PUBLIC

Score	Model1.1	Model1.2	Model1.3
Precision	N/A	1.0	0.99
Recall	N/A	1.0	1.0
F1	N/A	1.0	1.0

RESTRICTED

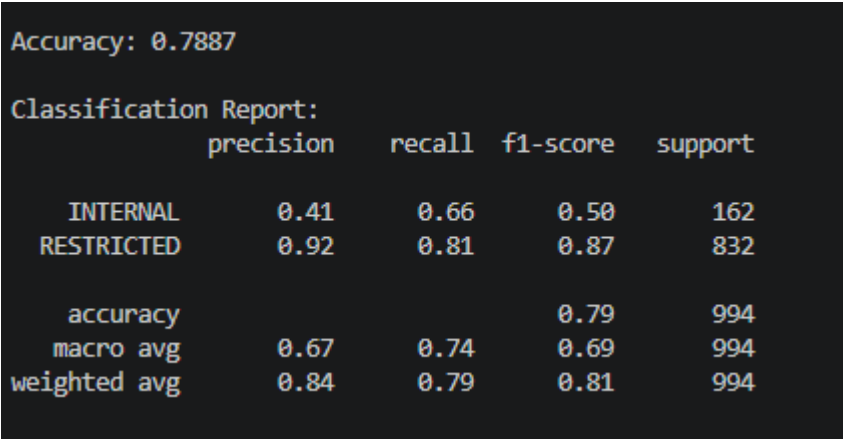
Score	Model1.1	Model1.2	Model1.3
Precision	0.96	0.96	0.95
Recall	0.75	0.79	0.89
F1	0.85	0.87	0.89

6.3.3 BioBERT Evaluation

BioBERT Model 1

Model 1 of BioBERT was developed to mirror the training environment of model1.1 of sklearn Logistic Regression. The dataset provided was identical being the unaltered MTSamples dataset, this provided a baseline for comparative analysis of models so that direct comparisons can be deduced between iterations. This provides a baseline for comparison between systems.

As can be seen from the results below BioBERT performed slightly worse overall indicating the same issues as discussed for Logistic Regression including the lack of a PUBLIC table and inconsistent INTERNAL results. It should be noted that BioBERT performed marginally worse than Logistic Regression on all INTERNAL metrics but outperformed in RECALL and F1 when comparing RESTRICTED metrics. These results further indicate an issue of quality and depth to the MTSamples dataset for classification.



```
Accuracy: 0.7887
Classification Report:
              precision    recall  f1-score   support

   INTERNAL         0.41         0.66         0.50         162
  RESTRICTED         0.92         0.81         0.87         832

 accuracy                   0.79         994
 macro avg         0.67         0.74         0.69         994
 weighted avg        0.84         0.79         0.81         994
```

Fig 6.6 biobert training 1

BioBERT Final Model

These results below indicate a significant improvement in BioBERTS performance when introducing synthetic data. The results of the PUBLIC dataset directly mirror Logistic Regression and BioBERT's first run, indicating purely synthetic data may not be viable for such a task. BioBERT compared to Logistic Regression shows enrichment of the dataset through introduction of synthetic data proves a near perfect recall and f1 score on the RESTRICTED category also showing substantial improvement in the precision of INTERNAL document.

The improvement on INTERNAL shows BioBERT is nearly always correct at classifying INTERNAL documentation but the recall score of 0.56 shows BioBERT is hesitant to deem a document INTERNAL, but when it does it's almost always correct. Interestingly sklearn produced an INTERNAL recall of 0.87 under the same conditions indicating BioBERT's conservative nature towards INTERNAL classification.

A RESTRICTED recall of 1.0 shows BioBERT consistently catches RESTRICTED documentation with no mistakes which could be seen as incredibly valuable in a clinical setting where missing a RESTRICTED document is the worst possible outcome.

It is interpreted that the 0.56 for INTERNAL recall shows BioBERT absorbing INTERNAL documentation into RESTRICTED.

The conclusion that can be drawn that under the same conditions BioBERT identifies RESTRICTED documentation at a far superior rate, yet Logistic Regression shows a greater power of discernment, this trade-off provides key insight into which approach would be more clinically viable, a conservative highly cautious or slightly less accurate more discerning approach.

Classification Report:				
	precision	recall	f1-score	support
INTERNAL	0.99	0.56	0.72	327
PUBLIC	1.00	0.99	1.00	120
RESTRICTED	0.87	1.00	0.93	952
accuracy			0.90	1399
macro avg	0.95	0.85	0.88	1399
weighted avg	0.91	0.90	0.89	1399

Fig 6.7 biobert training final

6.4 Model Evaluation In Deployment

To evaluate the real-world capabilities of these implementations, a controlled test was conducted using the cloud-based platform. This testing suite was conducted using the dataset created by Gemma 4 using Ollama as discussed in Phase 4 of Chapter 5.2.1. By using a different generator these tests evaluate the models ability to detect sensitivity characteristics.

The 15 documents were manually assigned expected classifications prior to testing based on their content which spans all 3 sensitivity categories as well as 2 deliberately ambiguous documents to test edge case handling. For the sake of the deployment testing, the confidence threshold discussed in Chapter 6.3.1 was disabled to ensure results outputted by the models was unaltered for accurate comparison.

Below is a table detailing each document, its expected classification result and summary of its content

File	Perceived Classification	Content
patient_discharge_cardiology_murphy.txt	RESTRICTED	Patient discharge notes containing name, diagnosis, medications prescribed, and follow up instructions
surgical_notes_appendectomy_2024.txt	RESTRICTED	Surgical operation notes, surgeon name, procedure details and post op instructions
mri_referral_neurology_dept.txt	RESTRICTED	MRI referral letter from a GP, patient details and symptoms
psychiatric_assessment_inpatient.txt	RESTRICTED	Patient complaint, patient name, mental state and risk assessment
oncology_chemotherapy_protocol.txt	RESTRICTED	Diagnosis, treatment protocol, medication dosage and future plans
staff_memo_infection_control.txt	INTERNAL	Plan for infection control protocol, no patient details
department_handover_notes_icu.txt	INTERNAL	ICU department handover notes, ward status, staffing levels, no patient or staff details
incident_report_equipment_failure.txt	INTERNAL	Internal incident report, no patient details, equipment failure, staff names and department
hr_memo_annual_leave_policy.txt	INTERNAL	HR memo about staff annual leave booking procedure
training_completion_record_fire_safety.txt	INTERNAL	Fire safety training procedures

visiting_hours_policy_2024.txt	PUBLIC	Updated visiting hours policy, no clinical or patient details, general information for public display
patient_rights_information_leaflet.txt	PUBLIC	Patient rights leaflet for public distribution
cafeteria_menu_week_april.txt	PUBLIC	Hospital Cafeteria menu
invoice_medical_supplies_q1.tx	Ambiguous	Invoice for medical supplies, contains supplier name, price and payment terms
referral_letter_physiotherapy.txt	Ambiguous	Referral letter to physiotherapist, minimal patient detail, administrative tone

Below is another table detailing average file size, word count and the total number of files tested, this will be cross referenced in the results and cost analysis.

File Descriptions
Average File Size: 1.8667kb Average File Word Count: 157.13 Total Files: 15

6.4.1 Results Evaluation and comparative analysis

Sklearn Performance Analysis

The following table represents the classification result produced by sklearn Logistic Regression for each of the 15 test documents.

The expected column denotes manually assigned ground truths based on document content, the classification column denotes the classification assigned by Logistic Regression and confidence denotes the model's reported probability for its chosen class.

File	Expected	Classification	Confidence	Result
patient_discharge_cardiology_murphy.txt	RESTRICTED	RESTRICTED	60.2%	Correct
surgical_notes_appendectomy_2024.txt	RESTRICTED	RESTRICTED	92.4%	Correct
mri_referral_neurology_dept.txt	RESTRICTED	RESTRICTED	52.5%	Correct
psychiatric_assessment_inpatient.txt	RESTRICTED	INTERNAL	67.7%	Wrong
oncology_chemotherapy_protocol.txt	RESTRICTED	INTERNAL	52.6%	Wrong
staff_memo_infection_control.txt	INTERNAL	RESTRICTED	65.1%	Wrong
department_handover_notes_icu.txt	INTERNAL	RESTRICTED	58%	Wrong
incident_report_equipment_failure.txt	INTERNAL	RESTRICTED	74.7%	Wrong
hr_memo_annual_leave_policy.txt	INTERNAL	RESTRICTED	68.5%	Wrong
training_completion_record_fire_safety.txt	INTERNAL	RESTRICTED	68.8%	Wrong
visiting_hours_policy_2024.txt	PUBLIC	RESTRICTED	74.6%	Wrong
patient_rights_information_leaflet.txt	PUBLIC	RESTRICTED	67.4%	Wrong
cafeteria_menu_week_april.txt	PUBLIC	RESTRICTED	64.7%	Wrong
invoice_medical_supplies_q1.txt	Ambiguous	RESTRICTED	77.6%	N/A
referral_letter_physiotherapy.txt	Ambiguous	INTERNAL	72%	N/A

Results

These results show a strong conservative bias by Logistic Regression. The model classified most documents as restricted regardless of content.

A conservative bias of over-restriction is not inherently problematic for internal documents which inherently contain some level of sensitive information. However, the misclassification of two Restricted documents and the Restricted classification of cafeteria_menu_week_april.txt is of greater concern. The full analysis and implications can be found in 6.4.2

Key Observations

Below is a brief overview of some of the key observations these in deployment tests yielded.

Over-classification	Logistic regression heavily biases towards restricted classification. Public documents such as the cafeteria menu were classified as restricted
Internal blind spot	Almost all internal documents were classified restricted showing a strong conservative trend
Confidence Range	52.6-92.4% showing a moderate confidence overall
Correct Classifications	Only 3/13 known classifications were correct

BioBERT Performance Analysis

The following table presents the classification results for BioBERT across the same 15 test documents as above in Sklearn. The same table structure is followed to preserve formatting and ensure direct comparability.

File	Expected	Classification	Confidence	Result
patient_discharge_cardiology_murphy.txt	RESTRICTED	RESTRICTED	99.7%	Correct
surgical_notes_appendectomy_2024.txt	RESTRICTED	RESTRICTED	99.9%	Correct
mri_referral_neurology_dept.txt	RESTRICTED	INTERNAL	99.4%	Wrong
psychiatric_assessment_inpatient.txt	RESTRICTED	RESTRICTED	54.6%	Correct
oncology_chemotherapy_protocol.txt	RESTRICTED	RESTRICTED	72.6%	Correct
staff_memo_infection_control.txt	INTERNAL	PUBLIC	73.8%	Wrong
department_handover_notes_icu.txt	INTERNAL	INTERNAL	81.7%	Correct
incident_report_equipment_failure.txt	INTERNAL	INTERNAL	94.6%	Correct
hr_memo_annual_leave_policy.txt	INTERNAL	INTERNAL	92.7%	Correct
training_completion_record_fire_safety.txt	INTERNAL	PUBLIC	99.6%	Wrong
visiting_hours_policy_2024.txt	PUBLIC	PUBLIC	99.9%	Correct
patient_rights_information_leaflet.txt	PUBLIC	INTERNAL	53.6%	Wrong
cafeteria_menu_week_april.txt	PUBLIC	PUBLIC	99.9%	Correct
invoice_medical_supplies_q1.txt	Ambiguous	RESTRICTED	99.55%	N/A
referral_letter_physiotherapy.txt	Ambiguous	INTERNAL	92.5%	N/A

Results

BioBERT demonstrates a significant improvement over sklearn in terms of accuracy in deployment correctly identifying 10/13 documents. The model shows a stronger understanding of domain and a Public to Internal distinction that Sklearn lacked. The misclassifications of BioBERT such as the fire safety and mri referral could be considered borderline misclassifications, this is explored more in depth in 6.4.2.

Key Observations

Classification Separation	Successful identification of public records
Internal recognition	Correctly classified internal documentation more than logistic regression demonstrating a more context aware approach
Confidence scores	Higher variance in confidence disparity 53.6-99.9% compared to logistic regression

Domain understanding	Confidential clinical records classified correctly as restricted.
Accuracy	9/13 correctly identified records

Comprehend Medical Performance Analysis

The following table presents the classification results for Comprehend Medical. As stated above all metrics, documents and formatting are consistent to ensure ease of comparison.

File	Expected	Classification	Confidence	Result
patient_discharge_cardiology_murphy.txt	RESTRICTED	RESTRICTED	100%	Correct
surgical_notes_appendectomy_2024.txt	RESTRICTED	RESTRICTED	100%	Correct
mri_referral_neurology_dept.txt	RESTRICTED	RESTRICTED	100%	Correct
psychiatric_assessment_inpatient.txt	RESTRICTED	RESTRICTED	100%	Correct
oncology_chemotherapy_protocol.txt	RESTRICTED	RESTRICTED	100%	Correct
staff_memo_infection_control.txt	INTERNAL	QUARANTINE	100%	Wrong
department_handover_notes_icu.txt	INTERNAL	QUARANTINE	98.3%	Wrong
incident_report_equipment_failure.txt	INTERNAL	RESTRICTED	100%	Wrong
hr_memo_annual_leave_policy.txt	INTERNAL	RESTRICTED	100%	Wrong
training_completion_record_fire_safety.txt	INTERNAL	RESTRICTED	99.9%	Wrong
visiting_hours_policy_2024.txt	PUBLIC	RESTRICTED	99.3%	Wrong
patient_rights_information_leaflet.txt	PUBLIC	QUARANTINE	28.8%	Wrong
cafeteria_menu_week_april.txt	PUBLIC	QUARANTINE	18.3%	Wrong
invoice_medical_supplies_q1.txt	Ambiguous	QUARANTINE	99.9%	Wrong
referral_letter_physiotherapy.txt	Ambiguous	QUARANTINE	43.6%	Wrong

Results

Comprehend Medical achieves complete 100% confidence of Restricted documentation but lacks the nuance to distinguish between Public and Internal successfully. This demonstrates the most extreme conservative bias of all 3 implementations. The full analysis of these results are discussed in depth in 6.4.2.

Key observations

Extreme over-classification	Almost all documents flagged Restricted
Public blind spots	Although internal recognition is a concern the lack of any public classification shows a strong trend to extreme classification
Confidence patterns	Extreme 100% scores on documentation show clear false positive trend

6.4.2 Deployment Evaluation

Sklearn in Deployment

Sklearn in deployment shows a significant regression from training performance. Of the 13 documents with tangible classification expectations sklearn only correctly identified 3, a deployment accuracy of only 23.07%. The model correctly identified all 3 Restricted clinical documents yet demonstrated a near total inability to identify Internal and Public documentation.

This behaviour remains consistent with training data, the model's Restricted classification domain consistently outperforms Internal and Public domains meaning the model, when confronted with unseen data errs towards Restricted classification.

Specifically, when confronted with public documentation the model classifies quite unexpectedly.

With regards to cafeteria_menu_week_april.txt which explicitly contains no medical documentation or PII the model still classified Restricted with a 64.7% confidence score.

This raises significant concern with regard to the practical deploy ability of Logistic Regression in a clinical environment. While benchmarking accuracies indicate strong overall performance, deployed blind tests demonstrate a strongly conservative and overestimating real world performance. This also presents further evidence to the overfitting concern from Llama 3.2 generated Public data with no real-world counterparts. This could imply that this model learned the writing style of Llama 3.2 rather than the data being presented, a real-world concern that further iterations should monitor closely and possibly amend by introducing more varying data to produce favourable results.

Another notable observation from the in-practice testing is memory consumption. When sklearn's Lambda function was defaulted to the 128mb memory limit it strongly suggests a bottlenecking concern that translates to worsening performance. When this memory limit was increased to the size of BioBERT's memory pool (3008mb) average inference time reduced from 2.9919s to 46.4ms a near 65x performance improvement. This suggests the cold start and inference time concerns were largely an infrastructural concern rather than a model limitation.

BioBERT in Deployment

BioBERT demonstrated a significantly stronger deployment performance than sklearn/ logistic regression, correctly identifying 10/13 documents with a definitive expected result. This places BioBERT at a 76.9% deployment accuracy. The model correctly identified all but one Restricted document, 3/5 Internal and 2/3 Public documents.

There is concern to be raised about the incorrectly classified documents. For example, mri_referral_neurology_dept.txt contains explicit PII such as name, date of birth and specific medical terminology yet BioBERT classified this document Internal with a 99.4% confidence score. This high confidence incorrect answer indicates perhaps BioBERT may not be classifying based off individual PII but instead based off document structure given the document reads as an email/ letter with minimal PII rather than a clinical diagnosis report.

This is coupled with the classification of staff_memo_infection_control.txt being misclassified as Public rather than Internal could be seen as a reasonable misclassification considering the contents of the document contain no patient PII and only minor elements of medical terminology.

The two ambiguous documents generated interesting results with `invoice_medical_supplies_q1.txt` being classified Restricted with 99.5% confidence. This could be seen as a conservative yet arguably sound logical conclusion considering the presence of information such as phone numbers, addresses and monetary amounts. The classification of `referral_letter_physiotherapy.txt` being Internal is consistent with previous classifications due to its largely administrative tone.

BioBERT in deployment seems to validate its training findings. The model demonstrates a strong Restricted detection and a tendency towards conservative classification which arbitrarily may be seen as favourable in the eyes of a clinical body as it tends to absorb Internal documentation into Restricted rather than demoting possibly sensitive documents.

Comprehend Medical in Deployment

Comprehend Medical presents perhaps the most interesting in deployment performance results. Although correctly identifying all Restricted documentation as Restricted the model was unable to identify any of the Internal or Public documentation correctly.

The pattern of misclassification is revealing of the Lambda invocation script surrounding it. The operation of this model functioned as expected returning PII and medical entities when called, the classification decision tree which mapped Comprehend Medicals outputs reflects a fundamental architectural limitation. The service which is designed for PII detection does not produce comparable results as a classification tool and the scores it returns are not directly comparable to classification probabilities.

Comprehend Medical also produced striking responses for PII and medical entity detection most notably in its classification of `cafeteria_menu_week_april.txt` citing “Wellness Bistro” as an instance of personally identifiable information. This result coupled with the low confidence score of 18.3% indicates possibly a fundamental limit for Comprehend Medical as a document classifier as its initial design was that of a PHI detector not a sensitivity classifier. When presented with documents containing neither clear PHI or medical terminology it produces a low confidence output and exits through its failsafe of quarantining the document.

This behaviour indicates a confirmation of the hypothesis proposed in 2.2.2 being that Comprehend Medicals blackbox nature and lack of fine tuning, while excellent at Restricting documents lacks the nuance “out of the box” to reliably detect the nuance between Internal and Public documentation, making it seemingly unsuitable to perform as a standalone classifier.

6.4.3 Performance Metrics Summary

A significant finding occurred during testing of sklearn's memory configuration. Increasing Lambda memory allocation from default 128mb to 3008mb reduced average inference time by roughly 65x. AWS by default allocates CPU memory according to allocated memory meaning a reduced memory pool leads to bottlenecking of classification. This provides an important implication that under the correct memory constraints sklearn becomes the substantially fastest inference model while maintaining a low cold start time.

BioBERT's cold start time represents the biggest limitation of the model, while its inference time is second lowest this large cold start period after inactivity cannot be understated. In a production environment this bottleneck would be limited due to the speed of clinical document generation. This increased compute time cannot be ignored as for each cold start completed further monthly costs are incurred. This inference time of 0.7012s can be seen as acceptable in its use case as classification is performed asynchronously.

Comprehend Medical's memory usage is notable as this score is not an average but a consistent memory usage across all classifications. It cannot at this time be determined if this would be the case for larger documents as the sample documents represented a roughly similar size structure. It can be assumed that due to the API structure of Comprehend Medical the inference time of 3.2906s represents network latency rather than computational time.

Model	Cold Start	Avg Inference	Avg Memory Used	Memory Allocated
Sklearn (128mb)	6.405s	2.9919s	128mb	128mb
Sklearn (3008mb)	2.986s	46.4286ms	272mb	3008mb
BioBERT	22.672s	0.7012s	966mb	3008mb
Comprehend Medical	2.374s	3.2906s	N/A	N/A

6.5 Results Evaluation and Comparative Analysis

Overall Performance

The comparative analysis of the three models' implementations across training and in deployment testing reveals a nuanced picture of the trade-offs between model complexity, accuracy and deployability in a clinical classification context.

Overall Performance Comparison

BioBERT presented as the strongest model in unseen classification testing with a 76.9% accuracy on Gemma4 documents. These results validate BioBERT's pre-training on a broad medical corpus. The models vast understanding of medical corpus is sufficiently broad to classify documents from a source it has never encountered. The three misclassifications could be seen as borderline cases where the document type itself could be seen as ambiguous.

The most striking finding of the total comparative analysis is the disparity between training accuracy and deployment tests of Logistic Regression. With a training accuracy of 85.6% but a deployment accuracy of 23.1% when performing unseen classification shows a fundamental weakness of the model.

This demonstrates a significant finding of the research being that benchmark accuracy on a held-out test set which derives from the same source as training data may be insufficient measure of real world deployability. The model learned to classify from MTSamples and Llama 3.2, but this did not translate to reliable classification in unseen tests.

Comprehend Medical demonstrated a binary classification pattern in deployment. With excellent classification results on Restricted documents but seeming unreliable in other aspects. Its perfect Restricted scores make a compelling argument for its viability as a PII detection tool but its inability to distinguish or provide context aware nuance renders it unsuitable as a classifier. It is possible this implementation could have provided a more fine-tuned approach to improving its classification abilities, this is further explained in Chapter 7.

6.5.1 GDPR Article 22

To fully satisfy the evaluation of the machine learning implementation of this project it should be noted that both the Logistic Regression and BioBERT implementations of this project satisfy the requirements of Article 22 of the GDPR stating that a data subject has the right to not be subject solely to automated decision making and by extension all automated decisions should be demonstratable and auditable. This is satisfied due to the fact that the training data, architecture and source code for confidence metric methodology are fully documented and reproducible. The basis of individual classification decisions can be dissected, challenged and overwritten. This cannot be said for Comprehend Medical and AWS Macie as their proprietary black box managed services cannot be audited sufficiently. For a production environment this implementation is a fundamental compliance consideration.

6.5.2 Cost Breakdown

Due to the free tier of AWS skewing per document results the use of Dashbird a per document pricing breakdown for each model was created, this provided further nuance to model choice and comparative analysis as it can be easily extrapolated the cost of each models deployment at scale. [56]

To gain meaningful insight from the inference numbers the calculation was extrapolated to 100,000 requests per month with memory allocation and duration matching the results gathered in 6.3.7 And then divided by 100,000 to provide a per document cost.

Model	Per 100,000	Per Document
Logistic Regression (128mb)	\$0.64	$\$6.4 \times 10^{-6}$
Logistic Regression (3008mb)	\$0.24	$\$2.4 \times 10^{-6}$
BioBERT	\$3.43	$\$3.43 \times 10^{-5}$
Comprehend Medical	\$0.71	$\$7.1 \times 10^{-6}$

This does not take into account Comprehend Medicals per inference pricing for use of its API which according to [57] would equate to (assuming an average 157.13 words per document equating to roughly 1135.3 characters per document) according to AWS Comprehend Medical pricing calculator each Comprehend Medical document would be priced at an additional \$0.14 per inference which is equal to a cost of \$14,000 per 100,000 documents.

This is troubling for Comprehend Medical costs as it reframes the comparative analysis. Comprehend Medical although providing perfect Restricted accuracy with no training overhead becomes prohibitively expensive at scale.

6.5.3 Research Question Evaluation

The central question of this project, whether machine learning is a viable approach to access control in a clinical environment to reduce GDPR risks is answered with affirmation.

Machine learning classification can be considered a viable option, but the choice of model and architecture is critical.

BioBERT Final Evaluation

BioBERT presents the best results across the categories discussed throughout Chapter 6. BioBERT presents a 76.9% deployment accuracy test while also boasting an 89.49% overall accuracy, combined with its explainability through fully demonstratable model fine tuning source code demonstrates that domain specific fine-tuned models can generalise and perform sufficiently for document classification.

Logistic Regression Final Evaluation

Logistic Regression, while computationally efficient both in inference time and cost performed poorly in deployment testing, this disparity between training and performance testing presents a stark contrast in results for clinical deployment and would likely require an extreme re-evaluation if it was to see clinical deployment.

Comprehend Medical Final Evaluation

Comprehend Medicals PII detection capabilities would be best understood as a complementary tool for classification rather than a standalone implementation. Its strength in detection could be combined with a trained model to deepen capabilities, but its cost and inability to perform as a classification tool make it inviable for individual deployments.

The cost analysis of 6.3.8 demonstrates that the additional computational overhead of BioBERT over Logistic Regression is marginal in Lambda deployments, making the performance advantage of the transformer approach justifiable at a clinical deployment scale. Comprehend Medicals per-document billing model scales unfavourably with document volume putting it out of reach for many clinical deployments.

6.6 Evaluate Project Management

6.6.1 Evaluate Project Plan

The project plan outlined in Chapter 4 proposed an incremental approach to development across 6 phases. The planned development process broadly succeeded in delivering all intended functionality, including all functional and non-functional requirements of the system.

The most significant unplanned event came in the form of the pivot from a locally deployed system to a fully cloud-native AWS architecture. Rather than derailing the project this architectural pivot provided greater nuance and depth to the original implementation while remaining faithful to the original goal.

The incremental method proved to be a great asset to development allowing for reiteration where needed that a more rigid structure such as the waterfall method may have struggled.

6.6.1 Evaluate Project Execution

Although project execution broadly went according to plan there were some unexpected deviations that required unplanned investigation.

Docker Containerisation

Containerisation was not in the original project plan and provided further external research to implement but was required to deploy the models as proposed.

Synthetic Data Generation

Initial project development did not account for the refusal of Llama 3.2 to generate the PII required for synthetic data generation. This unexpected issue required further analysis into prompt structure and further analysis of model choice to ensure results could be achieved.

Memory Configuration Discovery

During development a significant roadblock of memory and timeout management of Lambda functions was crucial to execution. During deployment testing of sklearn it was discovered that the default memory configuration of Lambda was restricting Logistic Regressions performance, this led to a further testing implementation with the increased memory functionality to ensure thorough and fair comparison.

Overall execution delivered the initial project plan successfully within the submission timeline.

6.7 Conclusions

This chapter presented a comprehensive evaluation of not only the cloud-based implementation, but the deep machine-learning comparative analysis implemented in this project. All functional test cases were passed successfully demonstrating full implementation according to requirements.

The ML evaluation section produced nuanced findings critical to the comparative analysis across the 6 implementations and deployment testing. These findings included the disparity between in practice performance compared to deployment testing of Logistic Regression and its extremely high classification speed when memory is increased.

BioBERT's demonstration of not only the least disparity between in practice and deployment testing but also the highest deployment accuracy of the 3 final models. These results validate BioBERT's extensive training and broad capabilities can be applied meaningfully when fine-tuned thoroughly.

The cost analysis section of this chapter also uncovered the additional cost associated with a larger model deployment vs a smaller model. BioBERT although significantly more accurate and nuanced in classification incurs much higher costs than comprehend medical.

These higher costs are entirely justifiable considering the much higher accuracy score of BioBERT.

7. Conclusions and Future Work

7.1. Introduction

This chapter presents the final conclusions of the research conducted throughout the project, directly addressing central research questions of whether machine learning assisted compliance is a viable approach to access control in a clinical environment to mitigate GDPR risks. The findings of the comparative analysis are summarised; limitations of current deployment are acknowledged and a plan for future work is presented.

7.2. Future Work

This system developed for this project is a strong system with well-defined direction for future iteration that could increase functionality and accuracy

The introduction of chained-classification pipelines touched upon in Chapter 6 could provide an added layer of nuance and protection to the classification pipeline. By chaining Comprehend Medicals detection to BioBERTs classification pipeline the system could increase classification confidence and auditability, producing a nuanced classification pipeline with PII detection and fully auditable classifications.

Dataset expansion and training refinement through interaction with real healthcare systems and documentation could also provide valuable training data to this system. Although the training pipeline was nuanced and deep the introduction of genuine PII and hospital documentation from Irish hospitals could greater strengthen the classification abilities of this system.

7.3. Conclusions

The comparative analysis of Chapter 6 produced nuanced findings that address the research question proposed for this project.

Sklearn Logistic Regression demonstrated that a classical machine learning implementation can achieve a strong benchmark performance with low latency for inference and negligible cost. Although the difference in results between training and in-practice testing are the most significant finding of this research. Benchmarking from hold-out testing appears as an insufficient metric for clinical deployability. This does not remove the promise the model provides. It's cost and inference speed cannot be neglected but the current training implementation proved insufficient to create a valid classifier.

BioBERT demonstrated the strongest results in terms of data processing in both training and in-practice testing. Its conservative bias towards Restricted classification holds merit in a clinical setting with large volumes of sensitive data. From a data protection perspective this overclassification is favourable to underclassification. Cost per inference is where this model lacks when compared to its sklearn counterpart but the increased cost should not be a deciding factor as this increased cost does not make it more expensive than tools such as Comprehend Medical and the favourable results it produces deployment shows its viability at clinical scale even at extra expense.

Comprehend Medical proved in testing that it's use could be more appropriate as a supplemental tool for compliance rather than a standalone implementation. Its heavy bias towards Restricting data is a methodology issue for how its detection mapped to classification. With refinement in future work this model could produce favourable results but as it currently stands the cost and Restricted bias makes it unfavourable for clinical deployment when compared to its counterparts.

The serverless AWS architecture developed for this system demonstrates that a GDPR compliant clinical document management platform with enterprise grade security is deployable at a cost accessible to organisations of any scale. Thus, directly addressing the gap in the market high-cost tools such as Purview and the lack of automated compliance solutions create.

Finally, to answer the central question of this report and system implementation, whether machine learning assisted compliance is a viable approach to reduce GDPR risks can be considered a success. BioBERTS results and cost implication demonstrate that ML-assisted document classification is technically feasible, implementable and produces tangible results in line with Article 22 of the GDPR.

References

- [1] 'Digital for Care - a new approach to healthcare'. Accessed: Apr. 02, 2026. [Online]. Available: <https://about.hse.ie/our-work/technology/digital-for-care-2030/>
- [2] B. O'Donovan, 'More than 470 legal actions against HSE over cyberattack', May 2024, Accessed: Apr. 02, 2026. [Online]. Available: <https://www.rte.ie/news/ireland/2024/0514/1448972-hse-cyber/>
- [3] 'A plan for digital health and social care', GOV.UK. Accessed: Apr. 13, 2026. [Online]. Available: <https://www.gov.uk/government/publications/a-plan-for-digital-health-and-social-care/a-plan-for-digital-health-and-social-care>
- [4] M. Kosinski, 'What Is a Data Breach? | IBM'. Accessed: Apr. 15, 2026. [Online]. Available: <https://www.ibm.com/think/topics/data-breach>
- [5] 'Art. 25 GDPR – Data protection by design and by default', General Data Protection Regulation (GDPR). Accessed: Apr. 17, 2026. [Online]. Available: <https://gdpr-info.eu/art-25-gdpr/>
- [6] Y. Hou *et al.*, 'Fine-tuning a local LLaMA-3 large language model for automated privacy-preserving physician letter generation in radiation oncology', *Front. Artif. Intell.*, vol. 7, Jan. 2025, doi: 10.3389/frai.2024.1493716.
- [7] 'Art. 32 GDPR – Security of processing', General Data Protection Regulation (GDPR). Accessed: Apr. 17, 2026. [Online]. Available: <https://gdpr-info.eu/art-32-gdpr/>
- [8] P. Khalate, S. Gite, B. Pradhan, and C.-W. Lee, 'Advancements and gaps in natural language processing and machine learning applications in healthcare: a comprehensive review of electronic medical records and medical imaging', *Front. Phys.*, vol. 12, Dec. 2024, doi: 10.3389/fphy.2024.1445204.
- [9] GitHub-Name, 'Windows 7 - Microsoft Lifecycle'. Accessed: Apr. 13, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/lifecycle/products/windows-7>
- [10] C. McCormack, 'Thousands of HSE computers rely on out-of-date software', Dec. 2020, Accessed: Apr. 13, 2026. [Online]. Available: <https://www.rte.ie/news/ireland/2020/1209/1183265-hse-technology/>
- [11] 'Tallaght University Hospital's Digital Transformation Journey with Evolve Cloud Platform'. Accessed: Apr. 13, 2026. [Online]. Available: <https://www.kainos.com/insights/success-stories/TUH-digital-transformation-journey>
- [12] National Institute of Standards and Technology (US), 'Advanced Encryption Standard (AES)', National Institute of Standards and Technology (U.S.), Washington, D.C., NIST FIPS 197-upd1, May 2023. doi: 10.6028/NIST.FIPS.197-upd1.
- [13] K. A. McKay and D. A. Cooper, 'Guidelines for the selection, configuration, and use of Transport Layer Security (TLS) implementations', National Institute of Standards and Technology, Gaithersburg, MD, NIST SP 800-52r2, Aug. 2019. doi: 10.6028/NIST.SP.800-52r2.
- [14] 'Art. 30 GDPR – Records of processing activities', General Data Protection Regulation (GDPR). Accessed: Apr. 17, 2026. [Online]. Available: <https://gdpr-info.eu/art-30-gdpr/>
- [15] 'Art. 17 GDPR – Right to erasure ("right to be forgotten")', General Data Protection Regulation (GDPR). Accessed: Apr. 17, 2026. [Online]. Available: <https://gdpr-info.eu/art-17-gdpr/>

- [16] 'Art. 22 GDPR – Automated individual decision-making, including profiling', General Data Protection Regulation (GDPR). Accessed: Apr. 17, 2026. [Online]. Available: <https://gdpr-info.eu/art-22-gdpr/>
- [17] 'Art. 5 GDPR – Principles relating to processing of personal data', General Data Protection Regulation (GDPR). Accessed: Apr. 17, 2026. [Online]. Available: <https://gdpr-info.eu/art-5-gdpr/>
- [18] 'General Data Protection Regulation (GDPR) – Legal Text', General Data Protection Regulation (GDPR). Accessed: Apr. 18, 2026. [Online]. Available: <https://gdpr-info.eu/>
- [19] 'Microsoft Purview Suite | Microsoft Security'. Accessed: Apr. 13, 2026. [Online]. Available: <https://www.microsoft.com/en-us/security/business/purview-suite-pricing>
- [20] 'Family Practices (GPs) Ireland'. Accessed: Apr. 09, 2026. [Online]. Available: <https://www.arcgis.com/sharing/rest/content/items/976a76eb20444684a2e437bcd48a1a>
- [21] 'National Healthcare Statistics 2024', gov.ie. Accessed: Apr. 16, 2026. [Online]. Available: <https://www.gov.ie/en/department-of-health/publications/national-healthcare-statistics-2024/>
- [22] A. Alhajeid, 'Microsoft 365 E5 Vs E3 Licensing Benefits – Which One Should You Choose', Wizard Cyber. Accessed: Apr. 13, 2026. [Online]. Available: <https://wizardcyber.com/microsoft-365-e5-vs-e3-licensing-benefits-which-one-should-you-choose/>
- [23] 'Office 365 E5 for enterprise | Microsoft 365'. Accessed: Apr. 18, 2026. [Online]. Available: <https://www.microsoft.com/en-ie/microsoft-365/enterprise/office-365-e5>
- [24] 'Microsoft Purview Suite | Microsoft Security'. Accessed: Apr. 18, 2026. [Online]. Available: <https://www.microsoft.com/en-ie/security/business/purview-suite-pricing>
- [25] 'St. Vincent's Healthcare Group', St. Vincent's University Hospital. Accessed: Apr. 18, 2026. [Online]. Available: <https://www.stvincents.ie/about-us/st-vincents-healthcare-group/>
- [26] 'Amazon Macie Pricing', Amazon Web Services, Inc. Accessed: Apr. 13, 2026. [Online]. Available: <https://aws.amazon.com/macie/pricing/>
- [27] 'Hospital In-Patient Enquiry (HIPE) | HIQA'. Accessed: Apr. 08, 2026. [Online]. Available: <https://www.hiqa.ie/areas-we-work/health-information/data-collections/hospital-patient-enquiry-hipe>
- [28] 'Art. 44 GDPR – General principle for transfers', General Data Protection Regulation (GDPR). Accessed: Apr. 17, 2026. [Online]. Available: <https://gdpr-info.eu/art-44-gdpr/>
- [29] *synthetichealth/synthea*. (Apr. 17, 2026). Java. Synthea. Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/synthetichealth/synthea>
- [30] J. Chen, D. Chun, M. Patel, E. Chiang, and J. James, 'The validity of synthetic clinical data: a validation study of a leading synthetic data generator (Synthea) using clinical quality measures', *BMC Med Inform Decis Mak*, vol. 19, no. 1, p. 44, Mar. 2019, doi: 10.1186/s12911-019-0793-0.
- [31] J. Walonoski *et al.*, 'Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record', *J Am Med Inform Assoc*, vol. 25, no. 3, pp. 230–238, Mar. 2018, doi: 10.1093/jamia/ocx079.
- [32] C. Peng *et al.*, 'A study of generative large language model for medical research and healthcare', *npj Digit. Med.*, vol. 6, no. 1, p. 210, Nov. 2023, doi: 10.1038/s41746-023-00958-w.

- [33] 'API Platform', OpenAI. Accessed: Apr. 19, 2026. [Online]. Available: <https://openai.com/api/>
- [34] F. Pedregosa *et al.*, 'Scikit-learn: Machine Learning in Python', Jun. 05, 2018, *arXiv*: arXiv:1201.0490. doi: 10.48550/arXiv.1201.0490.
- [35] A. Khabibrakhmanova, I. Alekseev, and R. Tazieva, 'Comparative Analysis of Learning Models for Solving Natural Language Processing Tasks', in *2024 International Russian Smart Industry Conference (SmartIndustryCon)*, Mar. 2024, pp. 56–61. doi: 10.1109/SmartIndustryCon61328.2024.10515815.
- [36] J. Lee *et al.*, 'BioBERT: a pre-trained biomedical language representation model for biomedical text mining', *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, Feb. 2020, doi: 10.1093/bioinformatics/btz682.
- [37] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds, Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423.
- [38] Y. Liu *et al.*, 'RoBERTa: A Robustly Optimized BERT Pretraining Approach', Jul. 26, 2019, *arXiv*: arXiv:1907.11692. doi: 10.48550/arXiv.1907.11692.
- [39] 'What is Amazon Comprehend Medical? - Amazon Comprehend Medical'. Accessed: Apr. 13, 2026. [Online]. Available: <https://docs.aws.amazon.com/comprehend-medical/latest/dev/comprehendmedical-welcome.html>
- [40] 'Medical Transcription Sample Reports & Examples | MTSamples'. Accessed: Apr. 18, 2026. [Online]. Available: <https://www.mtsamples.com/>
- [41] A. Jamil, M. K. Hanif, M. U. Sarwar, and M. I. Khan, 'An empirical evaluation of dimensionality reduction and class balancing for medical text classification', *Sci Rep*, vol. 16, no. 1, p. 815, Dec. 2025, doi: 10.1038/s41598-025-30537-w.
- [42] L. Almazaydeh, M. Abuhelaleh, A. Al Tawil, and K. Elleithy, 'Clinical Text Classification with Word Representation Features and Machine Learning Algorithms', *International Journal of Online and Biomedical Engineering (iJOE)*, vol. 19, pp. 65–76, Apr. 2023, doi: 10.3991/ijoe.v19i04.36099.
- [43] 'PubMed', PubMed. Accessed: Apr. 16, 2026. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/>
- [44] 'European Health Data Space Regulation (EHDS) - Public Health'. Accessed: Apr. 16, 2026. [Online]. Available: https://health.ec.europa.eu/ehealth-digital-health-and-care/european-health-data-space-regulation-ehds_en
- [45] ByteBot, 'Docker Hits 71% Adoption: Largest Tech Surge in 2025 Survey | byteiota'. Accessed: Apr. 10, 2026. [Online]. Available: <https://byteiota.com/docker-hits-71-adoption-largest-tech-surge-in-2025-survey/>
- [46] A. Paszke *et al.*, 'PyTorch: An Imperative Style, High-Performance Deep Learning Library', in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2019. Accessed: Apr. 14, 2026. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>

- [47] 'Welcome to Python.org', Python.org. Accessed: Apr. 18, 2026. [Online]. Available: <https://www.python.org/>
- [48] 'AWS SDK for Python - Boto3 - AWS', Amazon Web Services, Inc. Accessed: Apr. 14, 2026. [Online]. Available: <https://aws.amazon.com/sdk-for-python/>
- [49] 'Reliability Pillar - AWS Well-Architected Framework - Reliability Pillar'. Accessed: Apr. 17, 2026. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html>
- [50] 'Response Time Limits: Article by Jakob Nielsen', Nielsen Norman Group. Accessed: Apr. 17, 2026. [Online]. Available: <https://www.nngroup.com/articles/response-times-3-important-limits/>
- [51] G. Naidu, T. Zuva, and E. M. Sibanda, 'A Review of Evaluation Metrics in Machine Learning Algorithms', in *Artificial Intelligence Application in Networks and Systems*, R. Silhavy and P. Silhavy, Eds, Cham: Springer International Publishing, 2023, pp. 15–25. doi: 10.1007/978-3-031-35314-7_2.
- [52] A. Gholamy, V. Kreinovich, and O. Kosheleva, 'Why 70/30 or 80/20 Relation Between Training and Testing Sets: A Pedagogical Explanation', *Departmental Technical Reports (CS)*, Feb. 2018, [Online]. Available: https://scholarworks.utep.edu/cs_techrep/1209
- [53] '10 Usability Heuristics for User Interface Design', Nielsen Norman Group. Accessed: Apr. 16, 2026. [Online]. Available: <https://www.nngroup.com/articles/ten-usability-heuristics/>
- [54] Joint Task Force Interagency Working Group, 'Security and Privacy Controls for Information Systems and Organizations', National Institute of Standards and Technology, Sep. 2020. doi: 10.6028/NIST.SP.800-53r5.
- [55] C. Miller, T. Portlock, D. M. Nyaga, and J. M. O'Sullivan, 'A review of model evaluation metrics for machine learning in genetics and genomics', *Front Bioinform*, vol. 4, p. 1457619, Sep. 2024, doi: 10.3389/fbinf.2024.1457619.
- [56] 'Automating Serverless Best Practices with Dashbird's Serverless Well-Architected Insights | AWS Partner Network (APN) Blog'. Accessed: Apr. 15, 2026. [Online]. Available: <https://aws.amazon.com/blogs/apn/automating-serverless-best-practices-with-dashbird-serverless-well-architected-insights/>
- [57] 'Amazon Comprehend Medical Pricing', Amazon Web Services, Inc. Accessed: Apr. 15, 2026. [Online]. Available: <https://aws.amazon.com/comprehend/medical/pricing/>

Appendix A: System Analysis

File	Access Level	Confidence	Classified By	Timestamp	Status
visiting_hours_policy_2024.txt	RESTRICTED	99.3%	ML_COMPREHEND	2026-04-14T16:57:33.175189	CLASSIFIED
training_completion_record_fire_safety.txt	RESTRICTED	99.9%	ML_COMPREHEND	2026-04-14T16:57:28.736173	CLASSIFIED
surgical_notes_appendectomy_2024.txt	RESTRICTED	100.0%	ML_COMPREHEND	2026-04-14T16:57:26.684655	CLASSIFIED
staff_memo_infection_control.txt	QUARANTINE	100.0%	ML_COMPREHEND	2026-04-14T16:57:21.081819	CLASSIFIED
referral_letter_physiotherapy.txt	QUARANTINE	43.6%	ML_COMPREHEND	2026-04-14T16:57:17.101871	CLASSIFIED
psychiatric_assessment_inpatient.txt	RESTRICTED	100.0%	ML_COMPREHEND	2026-04-14T16:57:10.649701	CLASSIFIED
patient_rights_information_leaflet.txt	QUARANTINE	28.8%	ML_COMPREHEND	2026-04-14T16:57:03.658117	CLASSIFIED
patient_discharge_cardiology_murphy.txt	RESTRICTED	100.0%	ML_COMPREHEND	2026-04-14T16:57:00.890043	CLASSIFIED
oncology_chemotherapy_protocol.txt	RESTRICTED	100.0%	ML_COMPREHEND	2026-04-14T16:56:57.474651	CLASSIFIED
mri_referral_neurology_dept.txt	RESTRICTED	100.0%	ML_COMPREHEND	2026-04-14T16:56:52.180862	CLASSIFIED
invoice_medical_supplies_q1.txt	QUARANTINE	99.9%	ML_COMPREHEND	2026-04-14T16:56:47.084478	CLASSIFIED
incident_report_equipment_failure.txt	RESTRICTED	100.0%	ML_COMPREHEND	2026-04-14T16:56:42.145313	CLASSIFIED
hr_memo_annual_leave_policy.txt	RESTRICTED	100.0%	ML_COMPREHEND	2026-04-14T16:56:37.393011	CLASSIFIED
department_handover_notes_icu.txt	QUARANTINE	98.3%	ML_COMPREHEND	2026-04-14T16:56:34.813172	CLASSIFIED
cafeteria_menu_week_april.txt	QUARANTINE	18.3%	ML_COMPREHEND	2026-04-14T16:56:26.509945	CLASSIFIED

Fig A1 Comprehend Outputs Admin Dashboard

File	Access Level	Confidence	Classified By	Timestamp	Status
visiting_hours_policy_2024.txt	PUBLIC	99.9%	ML_BIOBERT	2026-04-14T17:11:12.111736	CLASSIFIED
training_completion_record_fire_safety.txt	PUBLIC	99.6%	ML_BIOBERT	2026-04-14T17:11:07.870468	CLASSIFIED
surgical_notes_appendectomy_2024.txt	RESTRICTED	99.9%	ML_BIOBERT	2026-04-14T17:11:04.891241	CLASSIFIED
staff_memo_infection_control.txt	PUBLIC	73.8%	ML_BIOBERT	2026-04-14T17:11:00.380724	CLASSIFIED
referral_letter_physiotherapy.txt	INTERNAL	92.5%	ML_BIOBERT	2026-04-14T17:10:55.126548	CLASSIFIED
psychiatric_assessment_inpatient.txt	RESTRICTED	54.6%	ML_BIOBERT	2026-04-14T17:10:49.302788	CLASSIFIED
patient_rights_information_leaflet.txt	INTERNAL	53.6%	ML_BIOBERT	2026-04-14T17:10:44.912090	CLASSIFIED
patient_discharge_cardiology_murphy.txt	RESTRICTED	99.7%	ML_BIOBERT	2026-04-14T17:10:40.163318	CLASSIFIED
oncology_chemotherapy_protocol.txt	RESTRICTED	72.6%	ML_BIOBERT	2026-04-14T17:10:36.039804	CLASSIFIED
mri_referral_neurology_dept.txt	INTERNAL	99.4%	ML_BIOBERT	2026-04-14T17:10:20.557536	CLASSIFIED
invoice_medical_supplies_q1.txt	RESTRICTED	99.5%	ML_BIOBERT	2026-04-14T17:10:16.736710	CLASSIFIED
incident_report_equipment_failure.txt	INTERNAL	94.6%	ML_BIOBERT	2026-04-14T17:10:13.578772	CLASSIFIED
hr_memo_annual_leave_policy.txt	INTERNAL	92.7%	ML_BIOBERT	2026-04-14T17:10:13.564548	CLASSIFIED
cafeteria_menu_week_april.txt	PUBLIC	99.9%	ML_BIOBERT	2026-04-14T17:10:13.473654	CLASSIFIED
department_handover_notes_icu.txt	INTERNAL	81.7%	ML_BIOBERT	2026-04-14T17:10:13.468937	CLASSIFIED

Fig A2 BioBERT Outputs Admin Dashboard

File	Access Level	Confidence	Classified By	Timestamp	Status
training_completion_record_fire_safety.txt	RESTRICTED	68.8%	ML_SKLEARN	2026-04-14T17:16:38.628145	CLASSIFIED
surgical_notes_appendectomy_2024.txt	RESTRICTED	92.4%	ML_SKLEARN	2026-04-14T17:16:34.628674	CLASSIFIED
staff_memo_infection_control.txt	RESTRICTED	65.1%	ML_SKLEARN	2026-04-14T17:16:30.788466	CLASSIFIED
referral_letter_physiotherapy.txt	INTERNAL	72.0%	ML_SKLEARN	2026-04-14T17:16:26.047332	CLASSIFIED
psychiatric_assessment_inpatient.txt	INTERNAL	67.7%	ML_SKLEARN	2026-04-14T17:16:22.169963	CLASSIFIED
patient_rights_information_leaflet.txt	RESTRICTED	67.4%	ML_SKLEARN	2026-04-14T17:16:18.429091	CLASSIFIED
patient_rights_information_leaflet.txt	RESTRICTED	67.4%	ML_SKLEARN	2026-04-14T17:16:15.668217	CLASSIFIED
patient_discharge_cardiology_murphy.txt	RESTRICTED	60.2%	ML_SKLEARN	2026-04-14T17:16:11.768402	CLASSIFIED
oncology_chemotherapy_protocol.txt	INTERNAL	52.6%	ML_SKLEARN	2026-04-14T17:16:08.528096	CLASSIFIED
mri_referral_neurology_dept.txt	RESTRICTED	52.5%	ML_SKLEARN	2026-04-14T17:16:05.167336	CLASSIFIED
invoice_medical_supplies_q1.txt	RESTRICTED	77.6%	ML_SKLEARN	2026-04-14T17:16:01.892358	CLASSIFIED
incident_report_equipment_failure.txt	RESTRICTED	74.7%	ML_SKLEARN	2026-04-14T17:15:57.468010	CLASSIFIED
incident_report_equipment_failure.txt	RESTRICTED	74.7%	ML_SKLEARN	2026-04-14T17:15:56.927349	CLASSIFIED
hr_memo_annual_leave_policy.txt	RESTRICTED	68.5%	ML_SKLEARN	2026-04-14T17:15:52.148637	CLASSIFIED
cafeteria_menu_week_april.txt	RESTRICTED	64.7%	ML_SKLEARN	2026-04-14T17:15:37.489357	CLASSIFIED
department_handover_notes_icu.txt	RESTRICTED	58.0%	ML_SKLEARN	2026-04-14T17:14:55.428945	CLASSIFIED

Fig A3 Sklearn Outputs Admin Dashboard

Appendix B: Prompts Used with Gen AI

B.1 Prompts Related to Coding

1. Can you assist in writing a function to upload a file to an AWS Lambda from a Javascript function?
2. Write a simple python shell script for prompting a local llama 3.2 installation.
3. Explain how downloading works from an AWS function to a web-based GUI
4. How can a Lambda function delete records from an S3 bucket?
5. Help write the error logging for the request to access a file in AWS through a check of Cognito Role
6. How can a lambda function return files to a Javascript function?
7. Search online for how to upload a machine-learning model to an AWS Lambda if Lambda doesn't have the dependencies by default
8. Im running into issues uploading a docker image to AWS how would I go about packaging, uploading and deploying in the cloud?
9. How can a Javascript function call an API gateway endpoint to receive a download URL?
10. How can I query two DynamoDB tables on each loading of a html page?
11. The confidence threshold for this classification pipeline does not function as intended, can you see any reason why this would not return anything at a confidence score below the threshold?
12. Can you search online for the Docker commands needed to upload an image to an AWS ECR?
13. Can you explain this error in upload to AWS ECR?
14. Is it possible to filter out returned results of a request for all file keys from S3 before it hits the UI or should it be written in the JavaScript function?
15. Why does Pytorch default to CPU in this code snippet? Cuda does not seem to be available even though my graphics card supports it
16. Can you assist in writing the Docker file for this container that will be uploaded to AWS ECR?
17. Why is Ollama returning this error when prompting an installed model through a Python function?
18. Can you help writing this function to merge 3 csv files together before preprocessing?
19. Write a javascript filtering function that filters returned file keys on the client side before displaying results to UI.
20. How can I extract the role from a Cognito JWT token to perform comparison between a table in DynamoDB
21. Is there a way to batch download all Lambda functions from an AWS project/ is it possible to write a script to perform that functionality?
22. How can I automate file download when this endpoint returns the download URL.
23. Can you explain how I would apply the metrics from this Logistic Regression implementation to a BioBERT implementation.

Note: When writing functions for AWS and machine-learning implementations, Copilot autocomplete was used to fill in functionalities such as error checking or completing repetitive tasks, no prompts were provided here but they were autocompleted.

Appendix C: Additional Code Samples

```
Training for 3 epochs...
Epoch 1/3 | Batch 0/249 | Loss: 0.7648
Epoch 1/3 | Batch 50/249 | Loss: 0.3536
Epoch 1/3 | Batch 100/249 | Loss: 0.2536
Epoch 1/3 | Batch 150/249 | Loss: 0.2357

Epoch 1 complete – Avg Loss: 0.3348 | Train Accuracy: 0.8215

Epoch 2/3 | Batch 0/249 | Loss: 0.4562
Epoch 2/3 | Batch 50/249 | Loss: 0.3515
Epoch 2/3 | Batch 100/249 | Loss: 0.1971
Epoch 2/3 | Batch 150/249 | Loss: 0.3014
Epoch 2/3 | Batch 200/249 | Loss: 0.2412

Epoch 2 complete – Avg Loss: 0.2726 | Train Accuracy: 0.8359

Epoch 3/3 | Batch 0/249 | Loss: 0.3092
Epoch 3/3 | Batch 50/249 | Loss: 0.1676
Epoch 3/3 | Batch 100/249 | Loss: 0.1689
Epoch 3/3 | Batch 150/249 | Loss: 0.2162
Epoch 3/3 | Batch 200/249 | Loss: 0.2845

Epoch 3 complete – Avg Loss: 0.2656 | Train Accuracy: 0.8341
```

Fig D1 BioBERT Training Outputs

Appendix D Testing Results:

Test Case ID: 1.0

Test Title: User Authentication

Description: Test that unauthenticated users cannot access the system and that authenticated users are presented with the correct interface

Pre-conditions: AWS Cognito user pool configured and active

Test 1 Successful Authentication

Step	Test Steps	Notes
1	User navigates to system using URL	Login screen presented
2	System presents login screen	System redirects user to Cognito login screen
3	User enters valid credentials	JWT token issued, interface displayed
4	Welcome message displays correct user email	Correct user email displayed

Test 2 Failed Authentication

Step	Test Steps	Notes
1	User navigates to system using URL	Login screen presented
2	System presents login screen	System redirects user to Cognito login screen
3	User enters invalid credentials	Error message displayed, login screen remains
4	Observe system response	

Result: Passed, both authentication flows proceeded as expected

Test Case ID: 2.0

Test Title: File Upload Via UI

Description: Test that authenticated users can successfully upload a file through the web interface

Pre-conditions

User authenticated, valid JWT token present in localStorage

Main Flow

Step	Test Steps	Notes
1	User selects file using file input	
2	User clicks upload button	
3	File is converted to base64 and sent to API Gateway with JWT token	
4	API Gateway validates JWT token via Cognito authoriser	EF 4.1
5	Upload Lambda receives file payload	
6	Lambda stores file in S3 bucket	
7	User receives success notification	

Error Flow

Step	Error Flow	Notes
4.1	JWT invalid or expired, API Gateway returns 401 (Unauthorised) user is presented with an error message	

Results: Passed

Test Case ID: 2.0

Test Title: File Upload Via UI

Description: Test that authenticated users can successfully upload a file through the web interface

Pre-conditions: User authenticated, valid JWT token present in localStorage

Test 1 Successful Authentication

Step	Test Steps	Notes
1	User navigates to system using URL	Login screen presented
2	System presents login screen	System redirects user to Cognito login screen
3	User enters valid credentials	JWT token issued, interface displayed
4	Welcome message displays correct user email	Correct user email displayed

Test 2 Failed Authentication

Step	Test Steps	Notes
1	User navigates to system using URL	Login screen presented
2	System presents login screen	System redirects user to Cognito login screen
3	User enters invalid credentials	Error message displayed, login screen remains
4	Observe system response	

Result: Passed, both authentication flows proceeded as expected

Test Case ID: 3.0

Test Title: File Listing

Description: Test that authenticated user can view files pertaining to their access level

Pre-conditions

AWS Cognito user pool configured and active, files present in S3
--

Main Flow

Step	Test Steps	Notes
1	User clicks refresh button on UI	
2	GET request sent to /listfiles endpoint using JWT token	
3	API Gateway validates token	
4	List files Lambda queries S3 for results	
5	Lambda returns file list with Metadata	
6	UI renders list in table format	EF 6.1

Error Flow

Step	Error Flow	Notes
6.1	No files present in bucket, UI returns files not found with accompanying error message	

Results: Passed

Test Case ID: 4.0

Test Title: File Searching

Description: Test that an authenticated user can search through the repository for files

Pre-conditions

User authenticated, valid JWT token present in localStorage

Main Flow

Step	Test Steps	Notes
1	User enters file name into search bar	
2	App.js performs a filter and returns file name	
3	User views file in filtered list	

Error Flow

Step	Error Flow	Notes

Results: Passed

Test Case ID: 5.0

Test Title: File Download

Description: Test that an authenticated user can download a file from the repository

Pre-conditions

User authenticated, valid JWT token present in localStorage, user has sufficient clearance, target file exists in s3.

Main Flow

Step	Test Steps	Notes
1	User navigates to system using URL	
2	User selects file and clicks download	2.1
3	System sends GET request to endpoint with JWT token	
4	System generates presigned URL	
5	URL returned to frontend	
6	Browser initiates file download	
7	Audit Log written to DynamoDB	

Error Flow

Step	Error Flow	Notes
2.1	File does not exist in S3	System returns error message

Results: Passed

Test Case ID: 6.0

Test Title: Sklearn Classification Pipeline

Description: Test that uploaded files trigger the sklearn classification Lambda and produce a valid classification.

Pre-conditions

File uploaded to S3, Lambda trigger for sklearn Lambda configured and active
--

Main Flow

Step	Test Steps	Notes
1	File uploaded to S3 bucket	
2	S3 PUT event triggers classification Lambda	
3	Lambda retrieves model.pkl from S3	
4	Lambda extracts text from uploaded file	EF 4.1
5	Vectorisation applied to text	
6	Logistic Regression model classifies document and returns classification and confidence score	
7	Classification result written to DynamoDB Classification Table	
8	Audit Log entry generated and written to DynamoDB audit log table	

Error Flow

Step	Error Flow	Notes
4.1	File content empty or unreadable, file marked QUARANTINE classification and writes error to CloudWatch logs for system administration.	

Results: Passed

Test Case ID: 7.0

Test Title: BioBERT Classification Pipeline

Description: Test that uploaded files trigger the BioBERT classification Lambda and produce a valid classification.

Pre-conditions

File uploaded to S3, Lambda trigger for BioBERT Lambda configured and active
--

Main Flow

Step	Test Steps	Notes
1	File uploaded to S3 bucket	
2	S3 PUT event triggers classification Lambda	
3	Lambda loads BioBERT model from container image in ECR	EF 3.1
4	Lambda extracts text from uploaded file	EF 4.1
5	Text Tokenised	
6	BioBERT model classifies document and returns classification and confidence score	
7	Classification result written to DynamoDB Classification Table	
8	Audit Log entry generated and written to DynamoDB audit log table	

Error Flow

Step	Error Flow	Notes
3.1	Container image too large for Lambda default timeout, increase timeout configuration in AWS GUI as well as memory allocation to 3gb	
4.1	File content empty or unreadable, file marked QUARANTINE classification and writes error to CloudWatch logs for system administration.	

Results: Passed

Test Case ID: 8.0

Test Title: AWS Comprehend Classification Pipeline

Description: Test that uploaded files trigger the Comprehend Medical classification Lambda and produce a valid classification.

Pre-conditions

File uploaded to S3, Lambda trigger for Comprehend Medical Lambda configured and active

Main Flow

Step	Test Steps	Notes
1	File uploaded to S3 bucket	
2	S3 PUT event triggers classification Lambda	
3	Lambda receives file content from S3	
4	Lambda submits text to Comprehend Medical API	EF 4.1
5	PII and medical entities detected	
6	Classification determined by Comprehend Medical	
7	Comprehend Medical returns classification result and confidence score	
8	Classification result written to DynamoDB Classification Table	
9	Audit Log entry generated and written to DynamoDB audit log table	

Error Flow

Step	Error Flow	Notes
4.1	File content empty or unreadable, file marked QUARANTINE classification and writes error to CloudWatch logs for system administration.	

Results: Passed

Test Case ID: 9.0

Test Title: Role Based Access Control: Access Granted

Description: Test that a user with sufficient clearance is granted access to a document and receives a pre-signed download URL.

Pre-conditions

User authenticated with sufficient clearance, appropriate clearance level document present in system with valid classification record in DynamoDB

Main Flow

Step	Test Steps	Notes
1	User requests document via UI	
2	GET request sent to /access endpoint with fileKey and JWT token	
3	API gateway validates token	EF 3.1
4	Access Control Lambda extracts user role from Cognito	
5	Lambda queries DynamoDB for classification result	
6	User role compared against access control matrix	
7	Access granted, lambda generates download URL with expiry time	
8	Access attempted logged to audit table	
9	URL returned to user	

Error Flow

Step	Error Flow	Notes
3.1	JWT token invalid or expired, system returns 401	

Results: Passed

Test Case ID: 10.0

Test Title: Role Based Access Control: Access Denied

Description: Test that a user with insufficient clearance is denied access to a document

Pre-conditions

User authenticated with insufficient clearance, appropriate clearance level document present in system with valid classification record in DynamoDB

Main Flow

Step	Test Steps	Notes
1	User requests document via UI	
2	GET request sent to /access endpoint with fileKey and JWT token	
3	API gateway validates token	
4	Access Control Lambda extracts user role from Cognito	
5	Lambda queries DynamoDB for classification result	
6	User role compared against access control matrix	
7	Access denied, lambda rejects user request	
8	Access attempted logged to audit table	
9	403 unauthorised returned to user	

Results: Passed

Test Case ID: 11.0

Test Title: Document Deletion (Right to Erasure)

Description: Test that an authorised user can permanently delete a document

Pre-conditions

User authenticated with sufficient role, document present in S3 with classification record
--

Main Flow

Step	Test Steps	Notes
1	User clicks delete button on file listing	
2	Confirmation of deletion dialogue box presented	EF 2.1
3	DELETE request sent to /delete API Gateway endpoint	
4	API Gateway validates token	
5	Delete lambda extracts user role from Cognito	
6	User role checked against delete permission	Ef 6.1
7	Document removed from S3 permanently	
8	DynamoDB classification table updated	
9	Deletion logged to Audit log table with timestamp and user role	

Error Flow

Step	Error Flow	Notes
2.1	User cancels confirmation dialogue, operation aborted	
6.1	Insufficient role, operation aborted	

Results: Passed

Test Case ID: 12.0

Test Title: Admin Dashboard

Description: Test that admin dashboard correctly displays all classification results and audit log entries

Pre-conditions

User authenticated with sufficient role, classification and audit entries present in DynamoDB

Main Flow

Step	Test Steps	Notes
1	Admin navigates to hidden endpoint admin.html	
2	Page calls loadDashboard on load	
3	GET request sent to /dashboard endpoint	
4	API gateway validates token	
5	Dashboard Lambda pulls from Classifications table	
6	Dashabord Lambda pulls form Audit Log table	
7	Summary statistics are calculated and returned	
8	Classification results rendered and displayed to user	
9	Audit logs rendered and displayed to user	

Results: Passed

Test Case ID: 13.0

Test Title: Human Override: Admin Reclassification

Description: Test that admin dashboard allows for manual reclassification of documents and writes to audit log

Pre-conditions

User authenticated with sufficient role, misclassified document present in system

Main Flow

Step	Test Steps	Notes
1	Admin views classification table on dashboard	
2	Admin selects misclassified document	
3	Admin selects new classification from dropdown	
4	Admin clicks reclassify	
5	Reason for classification prompted and entered	EF 5.1
6	POST request sent to /reclassify endpoint	
7	API Gateway validates token	
8	Reclassify Lambda verifies role	EF 8.1
9	DynamoDB classification record is updated	
10	Previous classification record is updated	
11	HUMAN_OVERRIDE audit entry written with entered reason and timestamp	
12	Dashboard displays updated record	

Error Flow

Step	Error Flow	Notes
5.1	Admin cancels reclassification, operation aborted	
8.1	Role insufficient, operation aborted	

Results: Passed

Test Case ID: 14.0

Test Title: Full End to End Upload and Classification Pipeline

Description: End to end test of full system interaction from document upload to classification

Pre-conditions

User authenticated, S3 active, S3 trigger active on classification Lambda

Main Flow

Step	Test Steps	Notes
1	User uploads document via UI	
2	File stored in S3 with AES256 encryption	EF 2.1
3	S3 PUT trigger fires classification Lambda	
4	Lambda classifies document and writes to DynamoDB	
5	Document returned classification results displayed to admin dashboard	

Error Flow

Step	Error Flow	Notes
2.1	S3 upload fails due to invalid file type	Error returned to user

Results: Passed

Test Case ID: 15.0

Test Title: Full Access Control Pipeline

Description: Test that end to end of complete document access pipeline including role validation

Pre-conditions

User authenticated, classified document present in S3

Main Flow

Step	Test Steps	Notes
1	User requests document	
2	Token validated by API Gateway	
3	User role extracted from Cognito	
4	Classification returned from DynamoDB	
5	Access decision made through comparison of classification to returned role	
6	Pre-signed URL generated and returned	
7	Access attempt recorded in AuditLog	
8	User downloads document via pre-signed URL	
9	URL expires after 5 minutes	

Error Flow

Step	Error Flow	Notes

Results: Passed